

Beyond Synthetic Noise: Deep Learning on Controlled Noisy Labels

Jiang, Lu et al. "Beyond Synthetic Noise: Deep Learning on Controlled Noisy Labels." *arXiv preprint arXiv:1911.09781* (2021).

Presenter

이재윤

Fundamental Team

고형권, 김동희, 김창연, 송현, 이민경

Contents

1. Noisy Label

2. Background

3. MentorMix

4. Experiments

1. Noisy Label





Dog



Dog



Cat?



Cat



Cat



Dog?

CLEAN



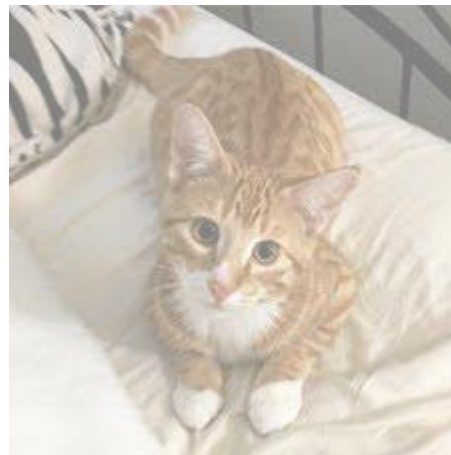
Dog

Cat



Dog

Cat



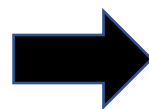
NOISY



Cat?

Dog?





Model



Preds



Loss

Dog

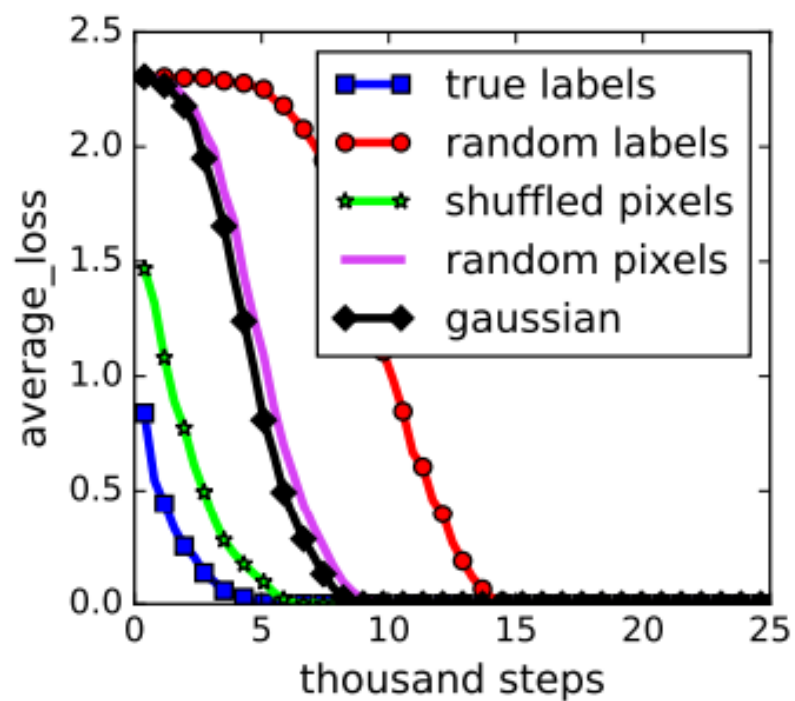
Cat

Dog
Label Y
Cat

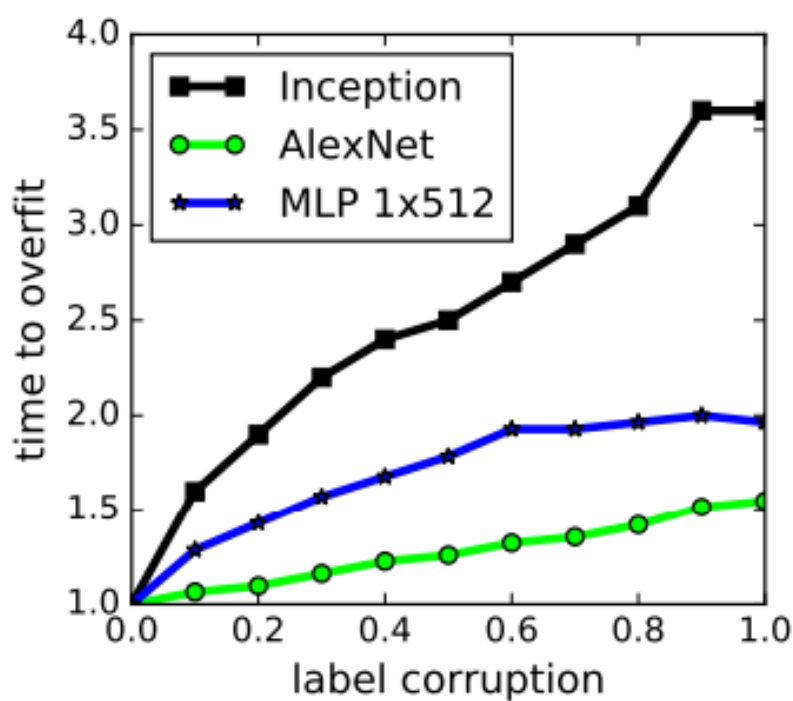
Cat

Dog

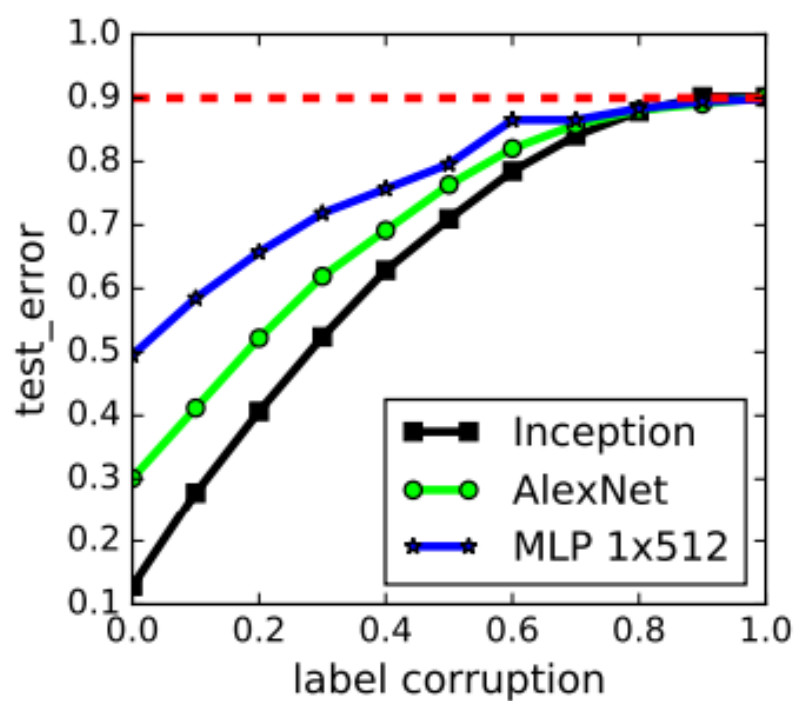




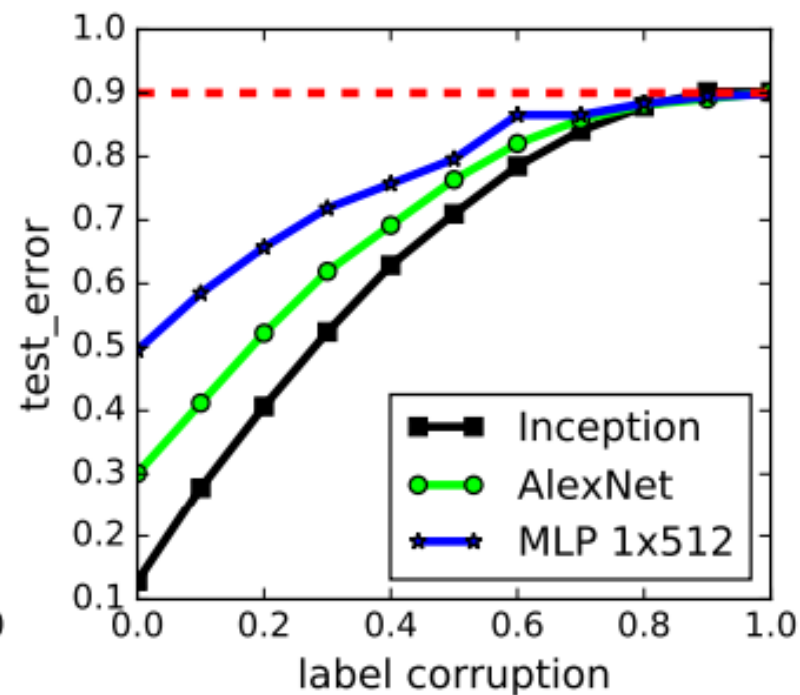
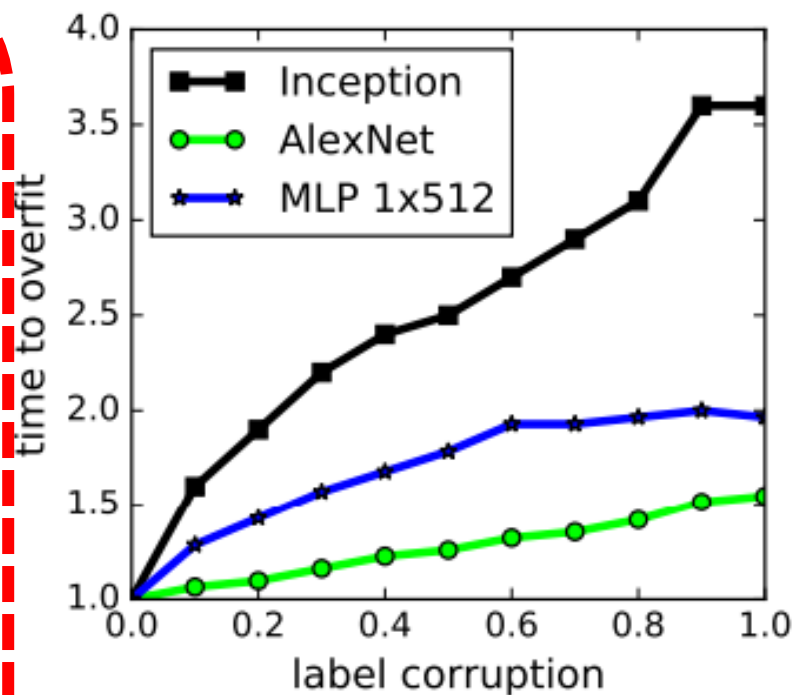
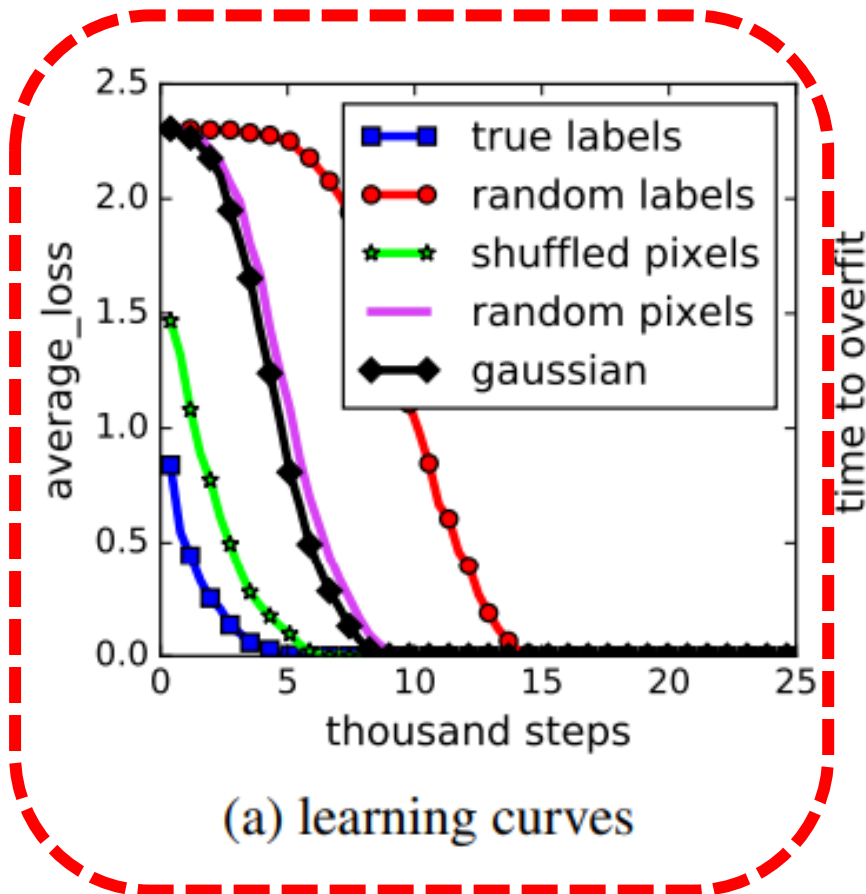
(a) learning curves

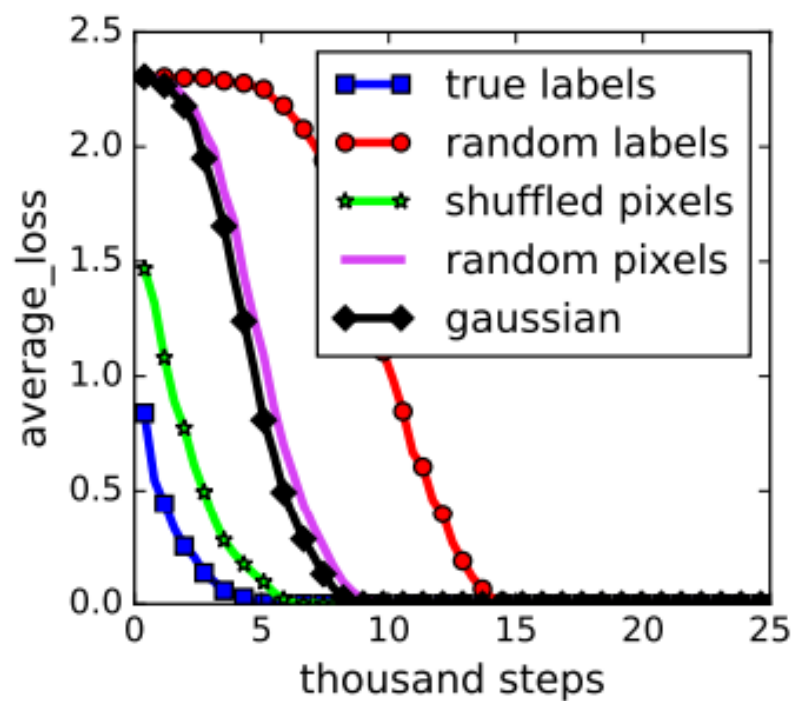


(b) convergence slowdown

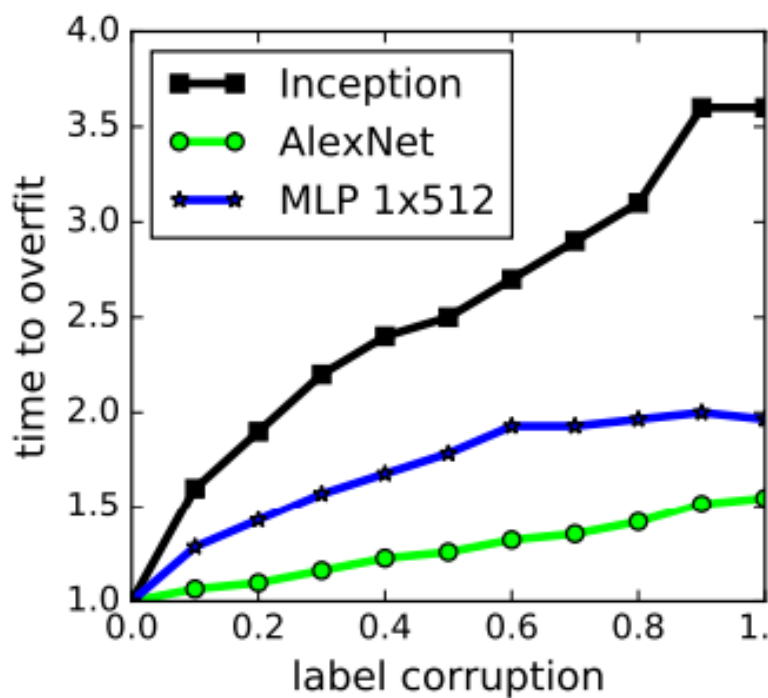


(c) generalization error growth

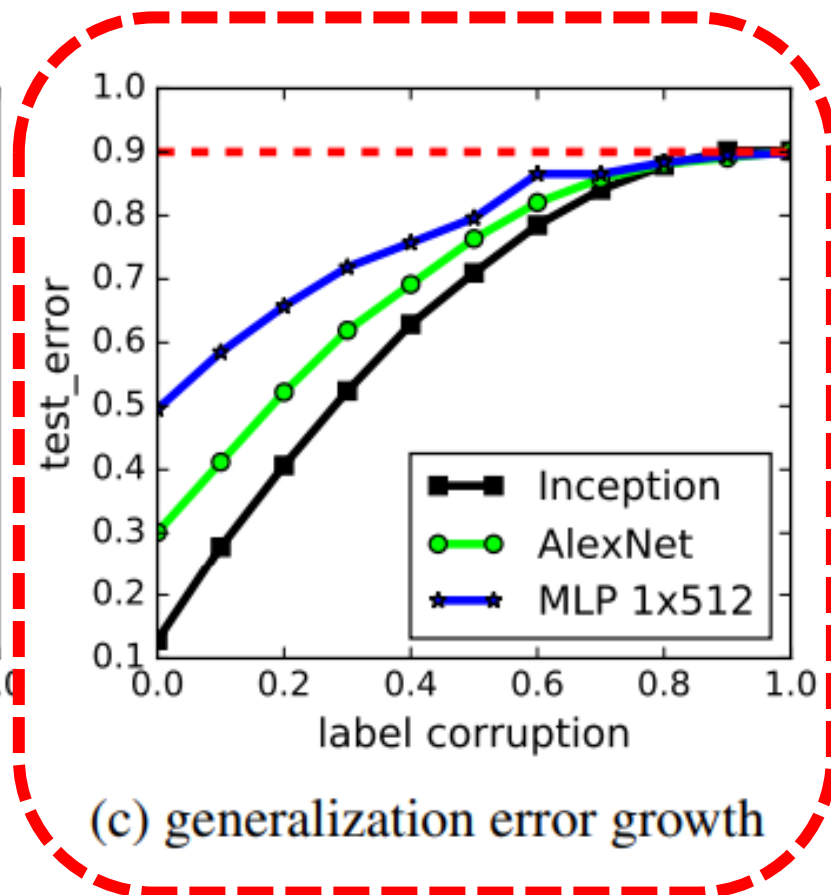




(a) learning curves



(b) convergence slowdown



(c) generalization error growth

2. Background

“*MentorMix:*

Minimize

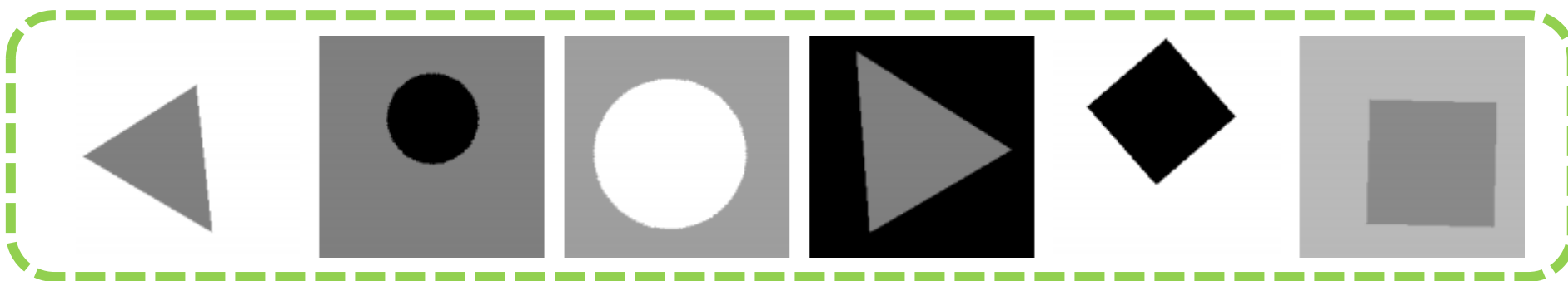
the ***empirical vicinal risk***

using ***curriculum learning***”

“*MentorNet* + *MixUp*:
Minimize
the *empirical vicinal risk*
using *curriculum learning*”

Curriculum Learning ?



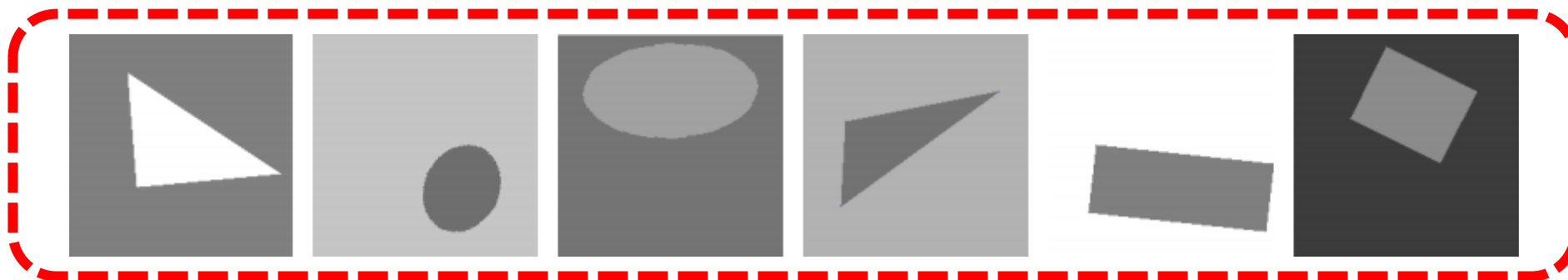


EASY





EASY



HARD

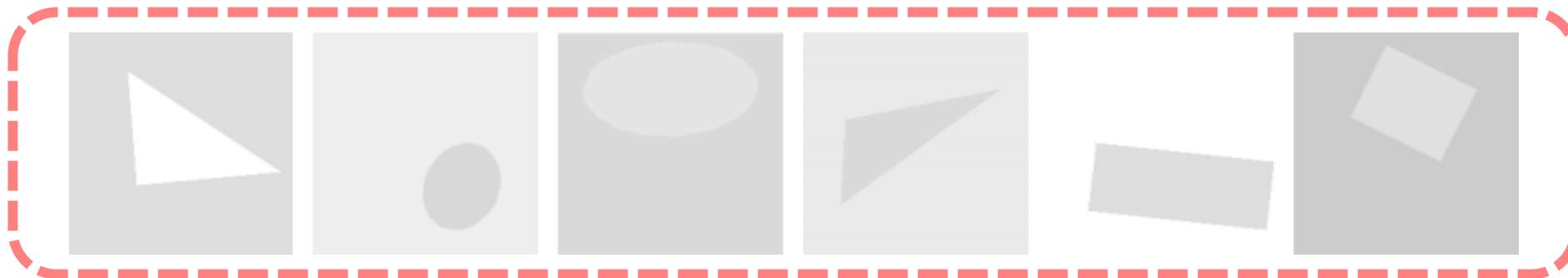


Faster

EASY

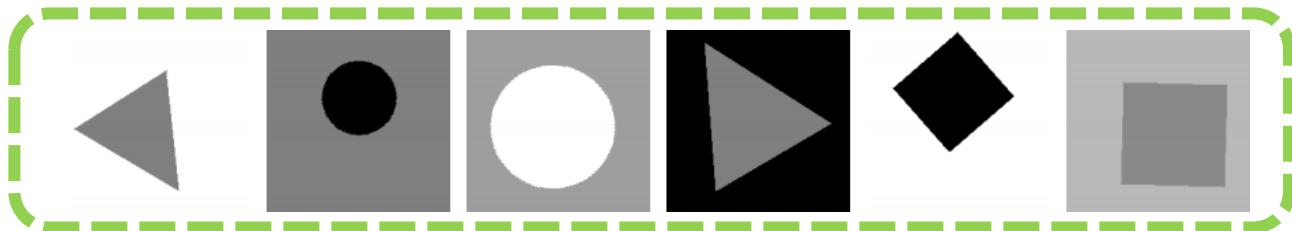


Better

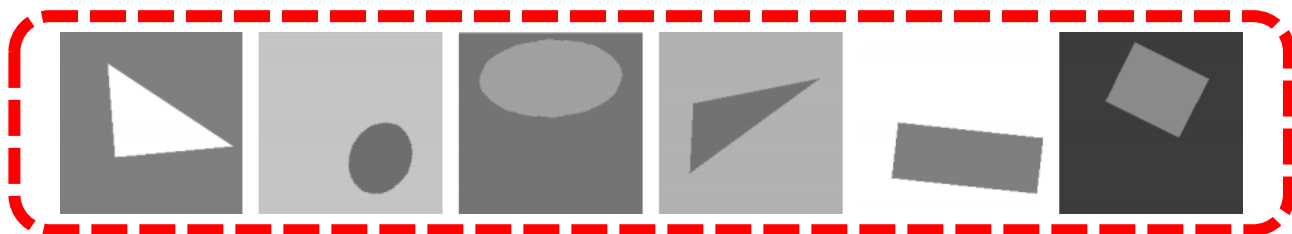


HARD

Case 1



EASY



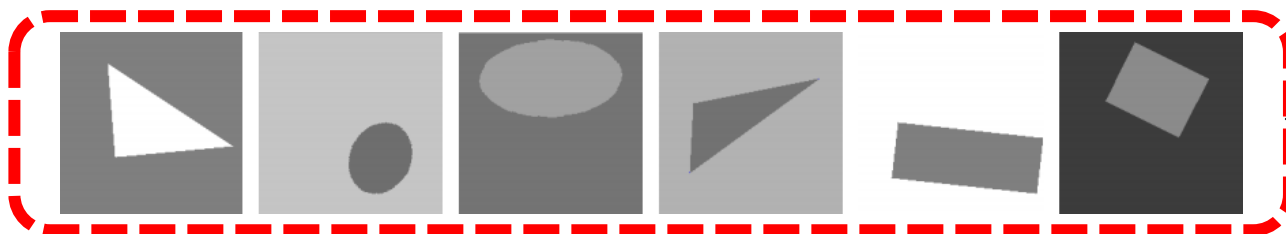
HARD



Case 1



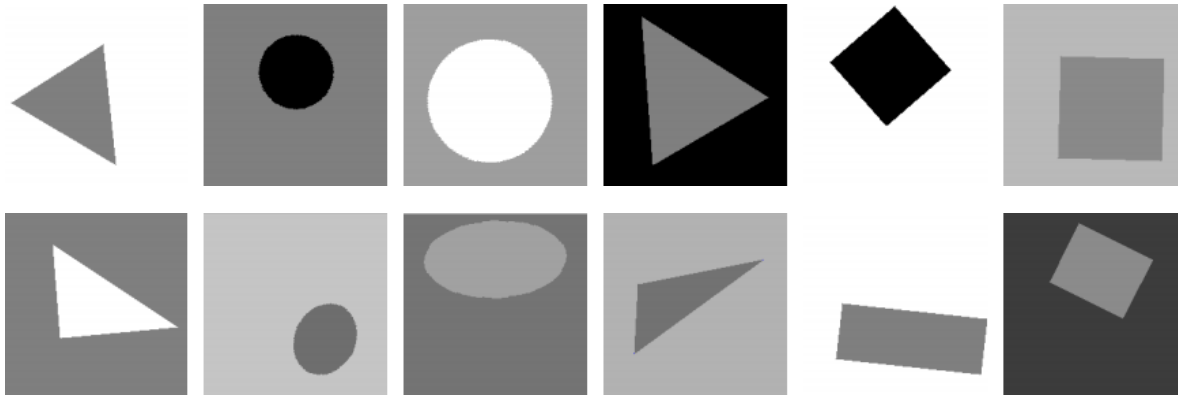
EASY



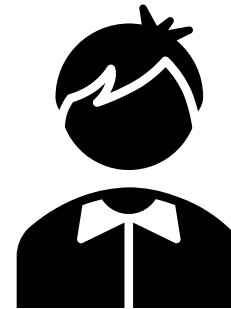
HARD



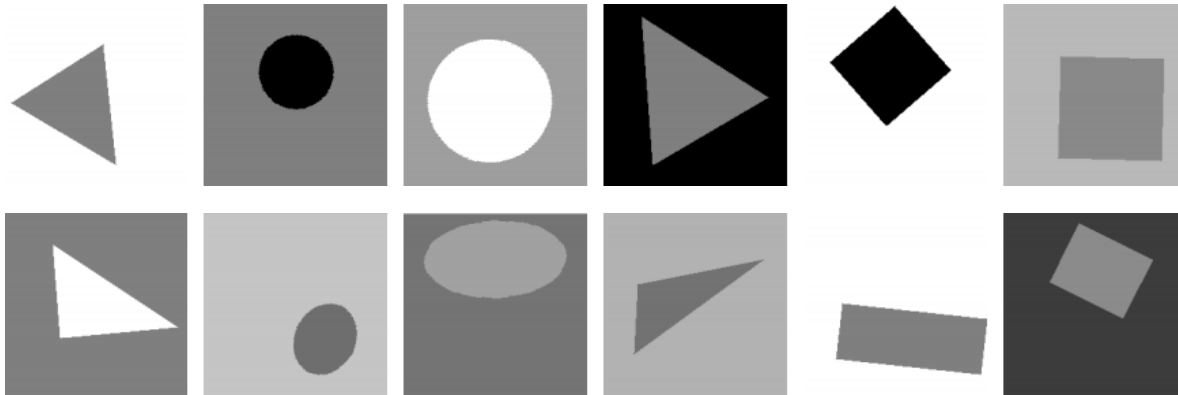
Case 2



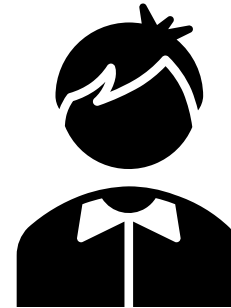
*Which one is
easy/**hard**?*



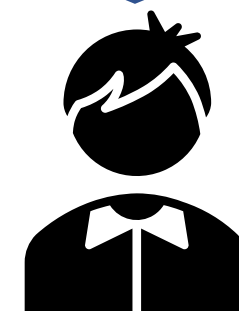
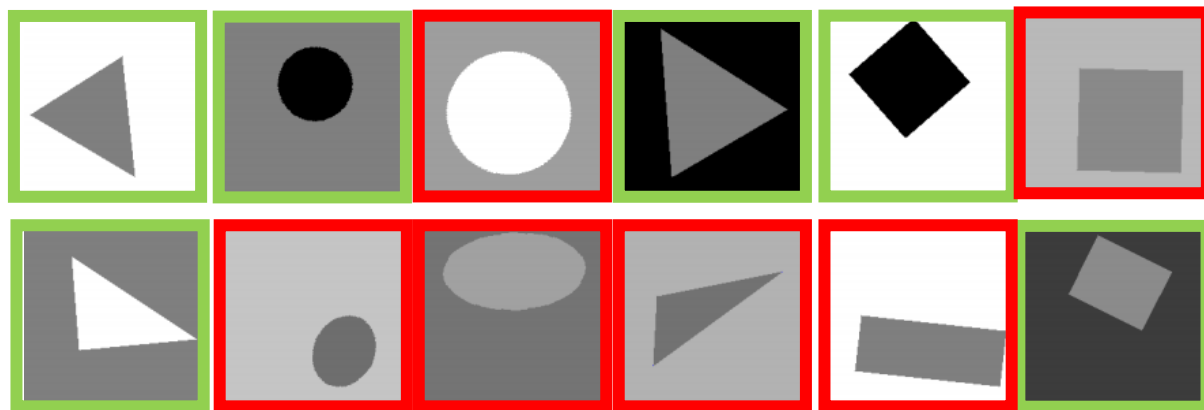
Case 2



*Let's just solve
everything!*

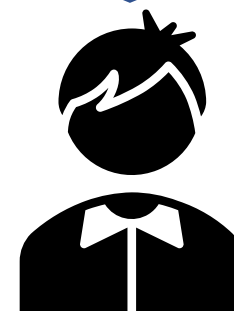
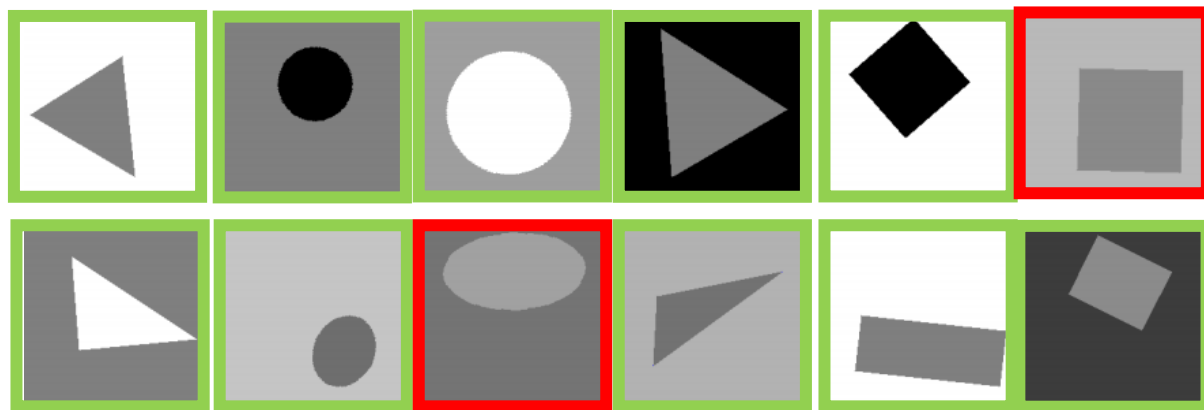


Case 2



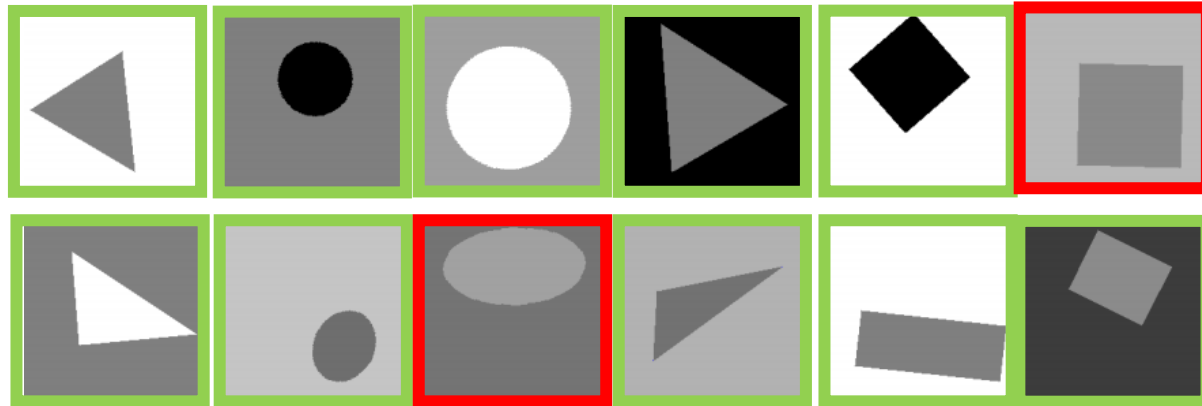
*/ think they are
easy*

Case 2

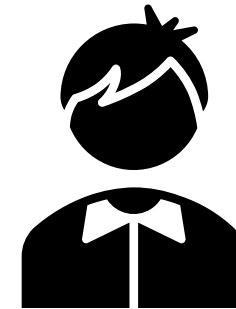


*/ think they are
easy*

Case 2

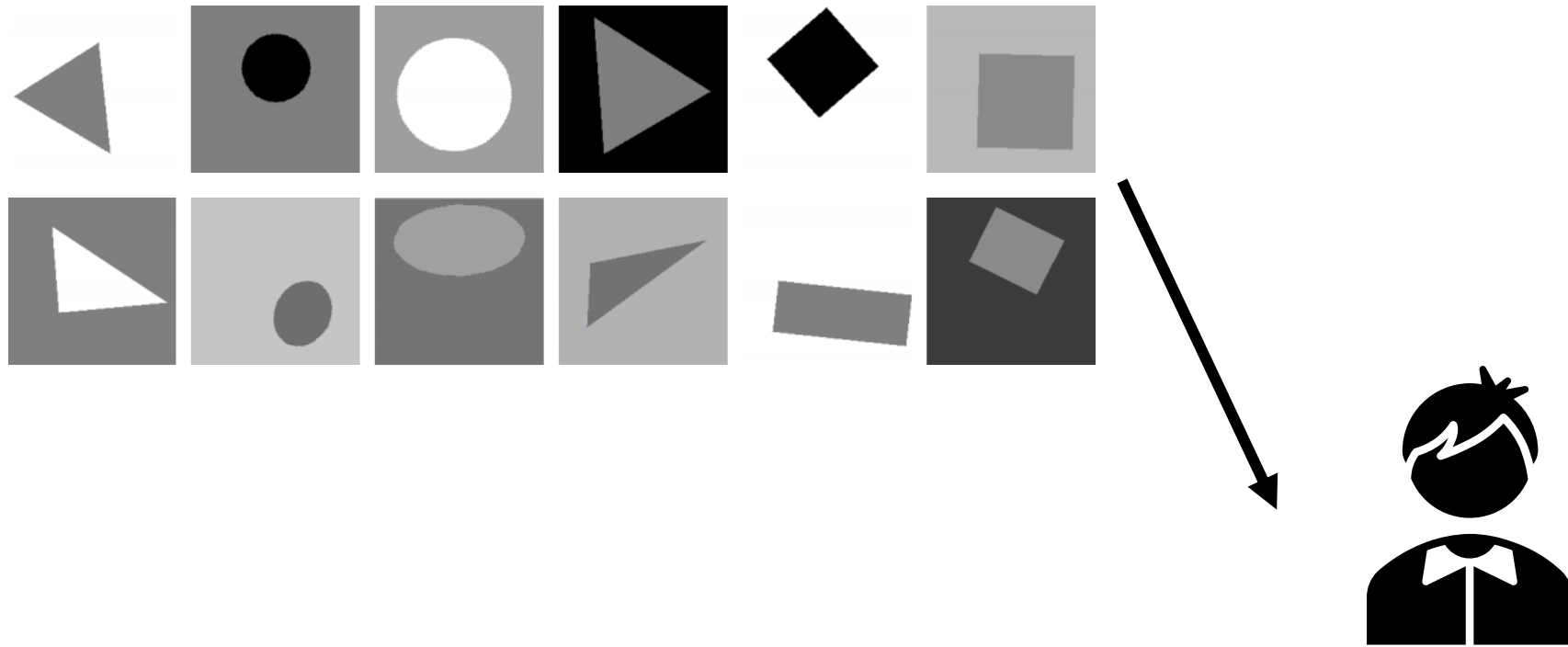


*/ think they are
easy*

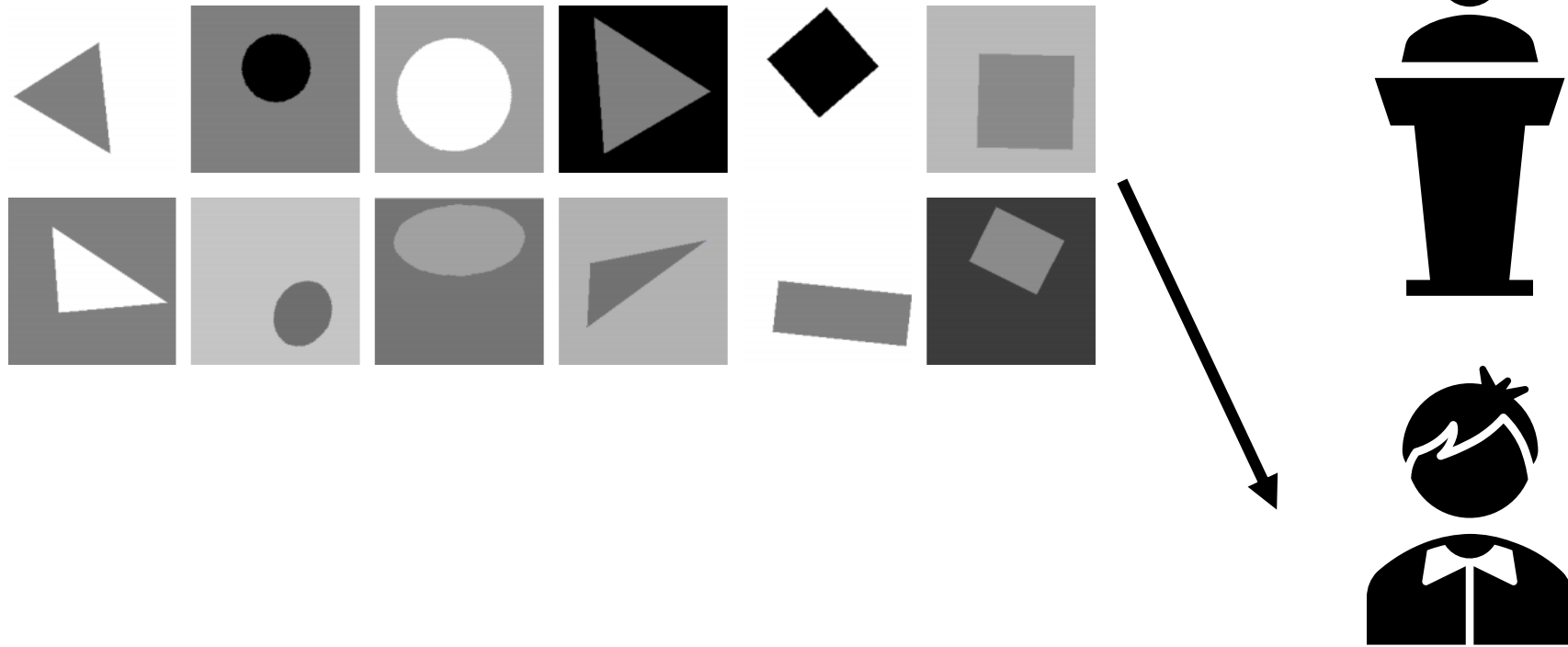


“Self – Paced Learning”

Case 3 (MentorNet)

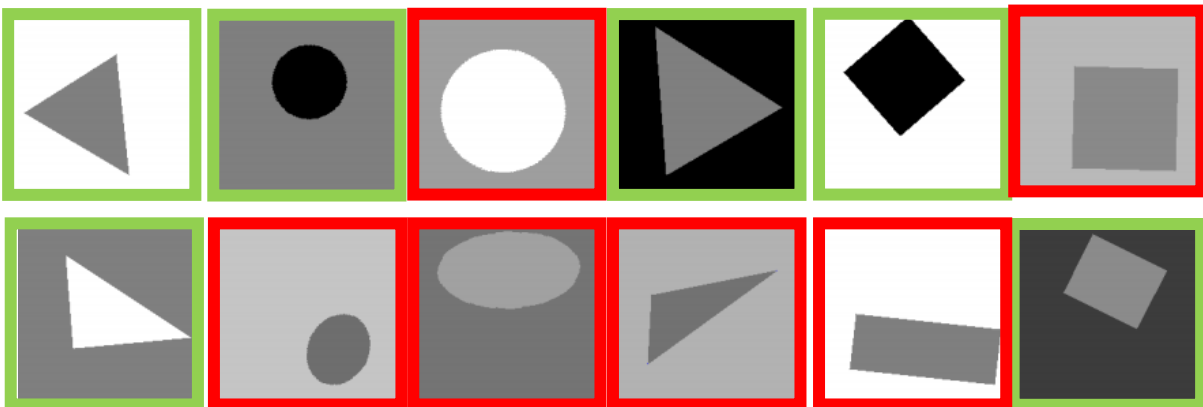
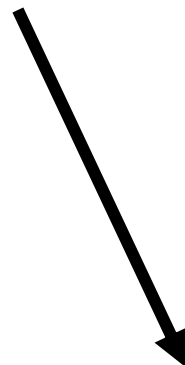


Case 3 (MentorNet)



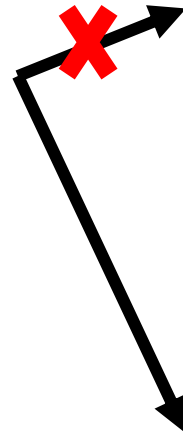
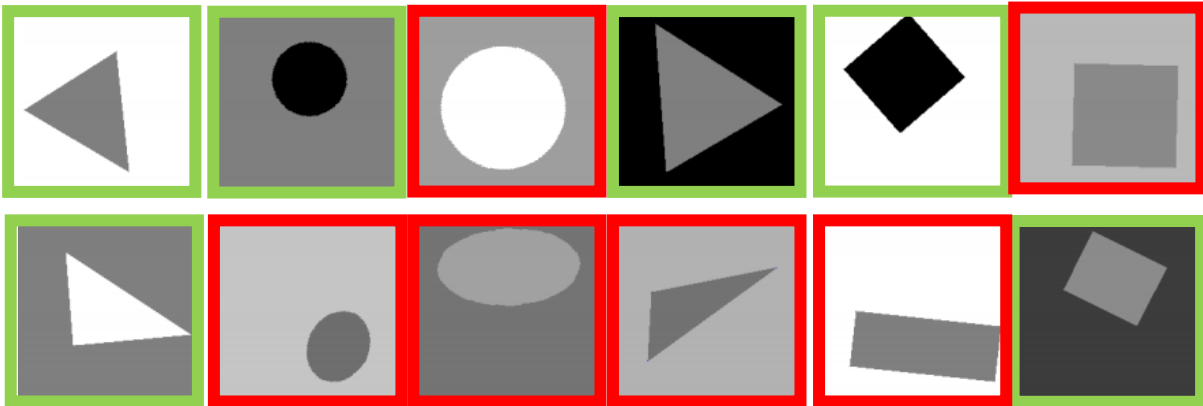
Case 3 (MentorNet)

*I think they are
easy*



Case 3 (MentorNet)

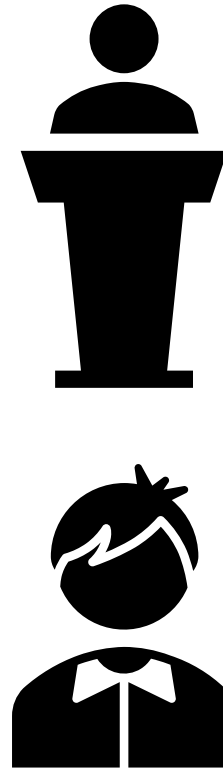
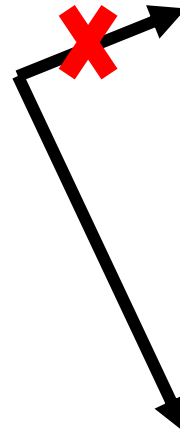
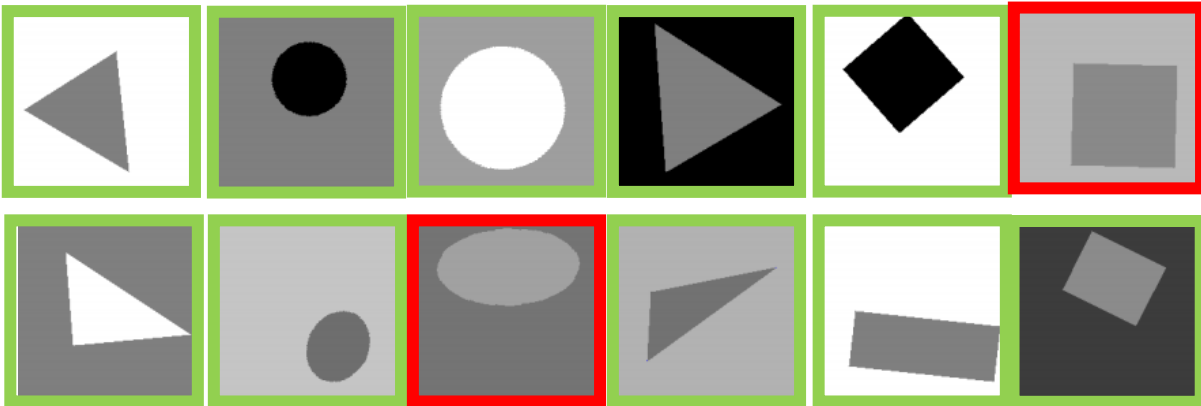
*/ think they are
easy for student*



- Student's interpretation
- Current Score
- Last Score
- Degree of progress

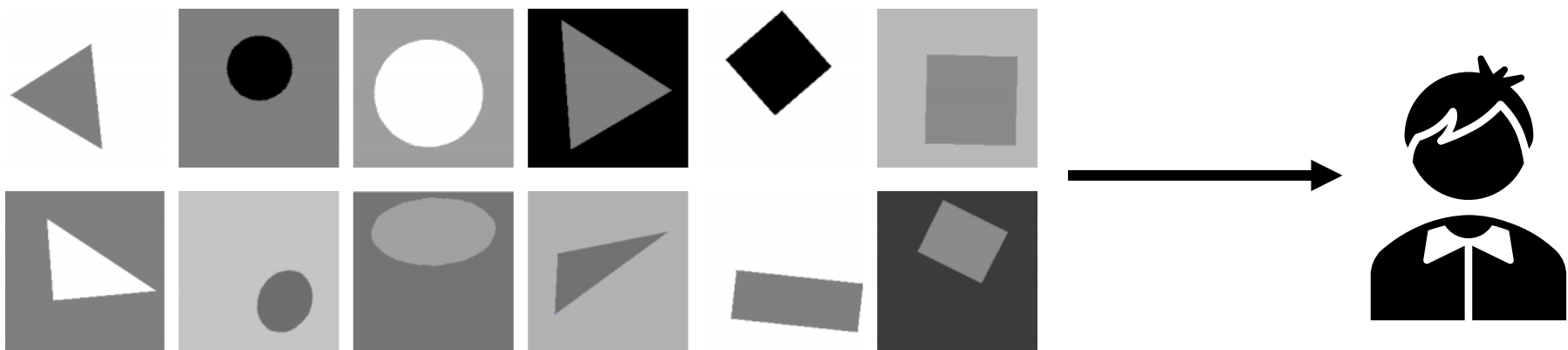
Case 3 (MentorNet)

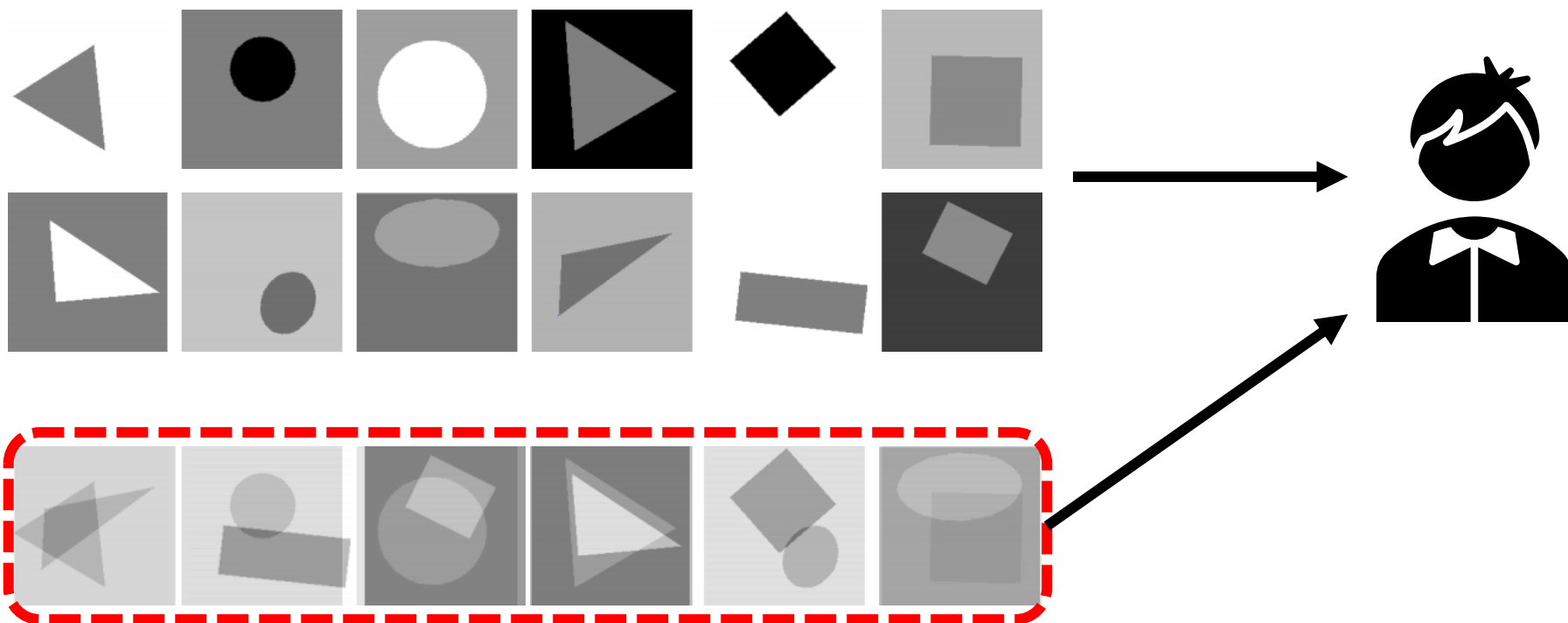
*/ think they are
easy for student*

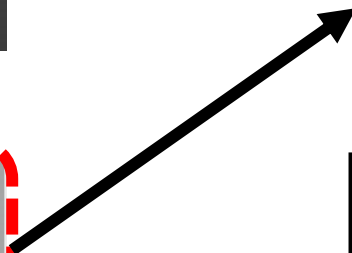
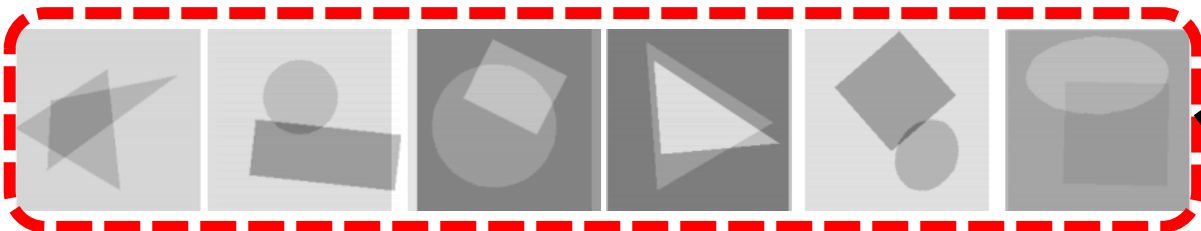
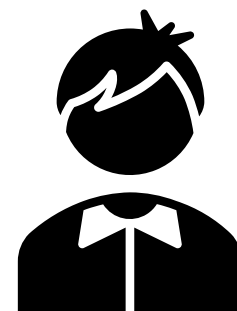
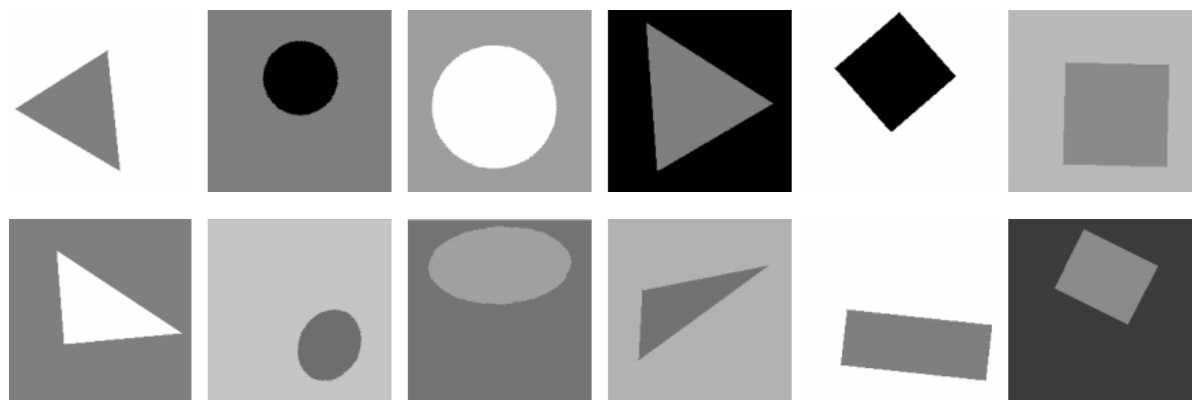


- Student's interpretation
- Current Score
- Last Score
- Degree of progress

Mixup?







Prevent Overfitting

3. MentorMix

Easiness in Noisy Label



Dog



Dog



Cat



Cat



Cat



Dog



Easiness in Noisy Label



Dog



Dog



Cat



Cat



Cat



Dog



Easiness in Noisy Label



Dog



Dog



Cat



Cat



Cat



Dog



Easiness in Noisy Label



Dog



Dog



Cat



Cat



Cat



Dog

*It is **hard***



It's Cat!!



Easiness in Noisy Label



Dog



Dog



Cat



Cat



Cat



Dog



Easiness in Noisy Label



Dog



Dog



Cat



Cat



Cat



Dog

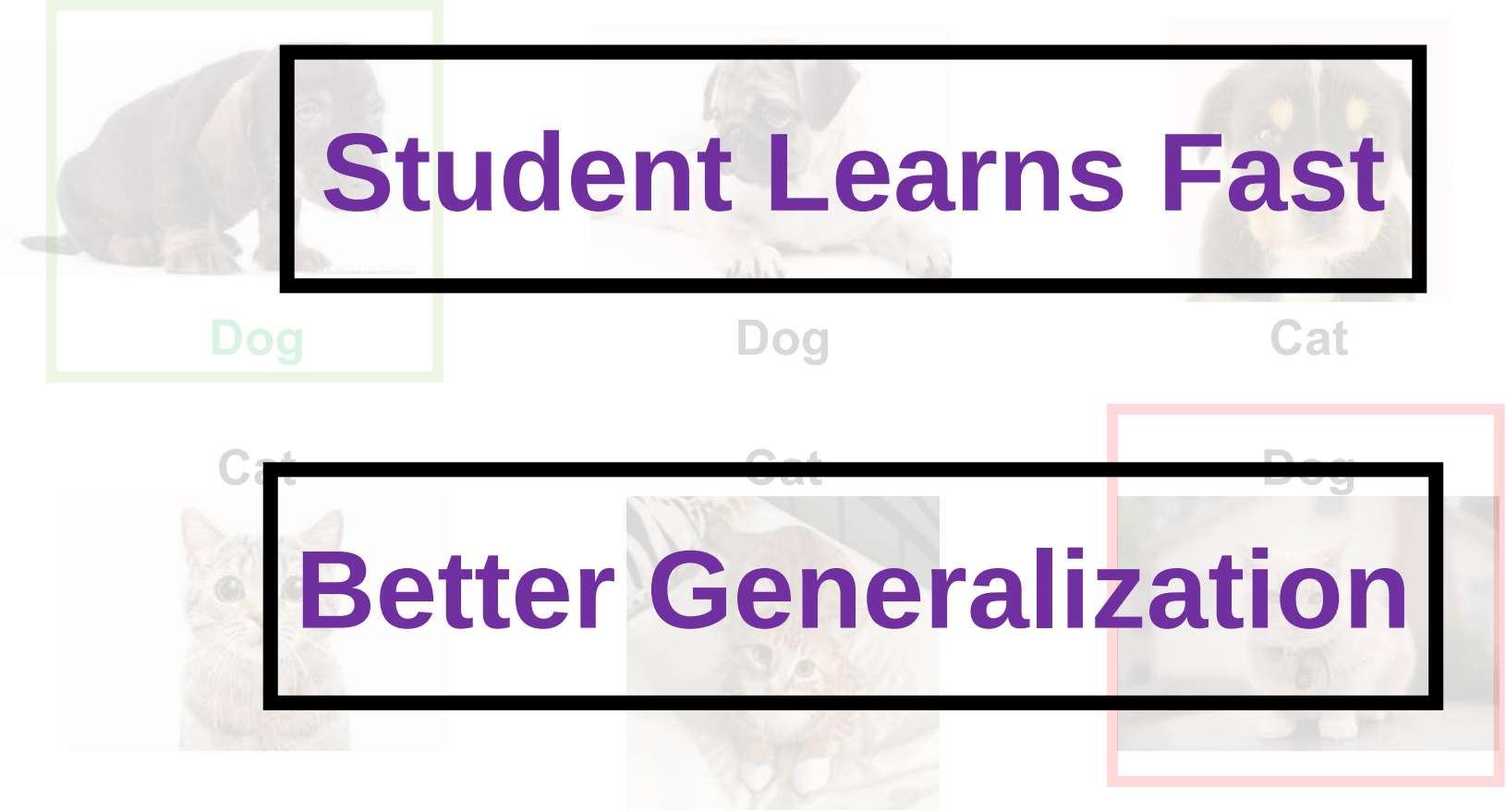
*It is **hard***



It's Cat!!

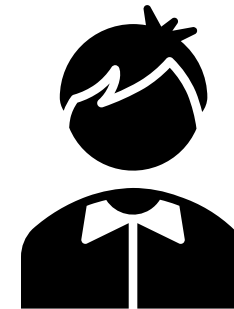


Easiness in Noisy Label



Student Learns Fast

Better Generalization



Implementaion

MentorMix Implementation

$$F(v, w)$$

- n : number of training example
- $v \in [0,1]^n$: latent weight variable
- w : weight of **student** model
- $F(v, w)$: Loss function
- $g_s(x_i; w, y_i)$: prediction of DNN
- G : Curriculum
- γ : loss threshold

MentorMix Implementation

$$F(v, w)$$

$$= \frac{1}{n} \sum_{i=1}^n v_i l(g_s(x_i; w), y_i) + \theta \|w\|_2^2 - G(v; \gamma)$$

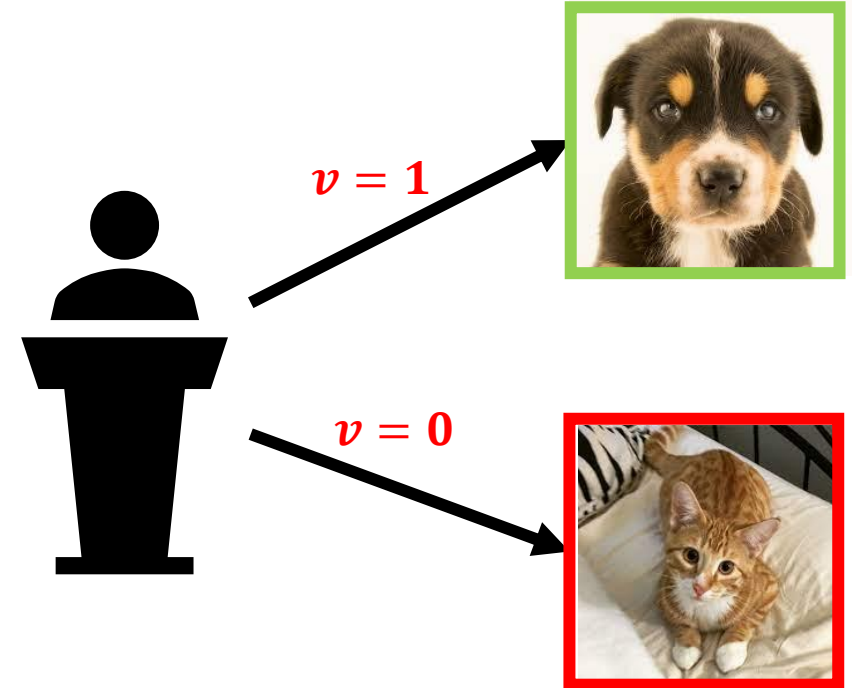
Cross Entropy Loss

- n : number of training example
- $v \in [0,1]^n$: latent weight variable
- w : weight of **student** model
- $F(v, w)$: Loss function
- $g_s(x_i; w, y_i)$: prediction of DNN
- G : Curriculum
- γ : loss threshold

MentorMix Implementation

$$F(v, w) = \frac{1}{n} \sum_{i=1}^n v_i l(g_s(x_i; w), y_i) + \theta \|w\|_2^2 - G(v; \gamma)$$

Cross Entropy Loss

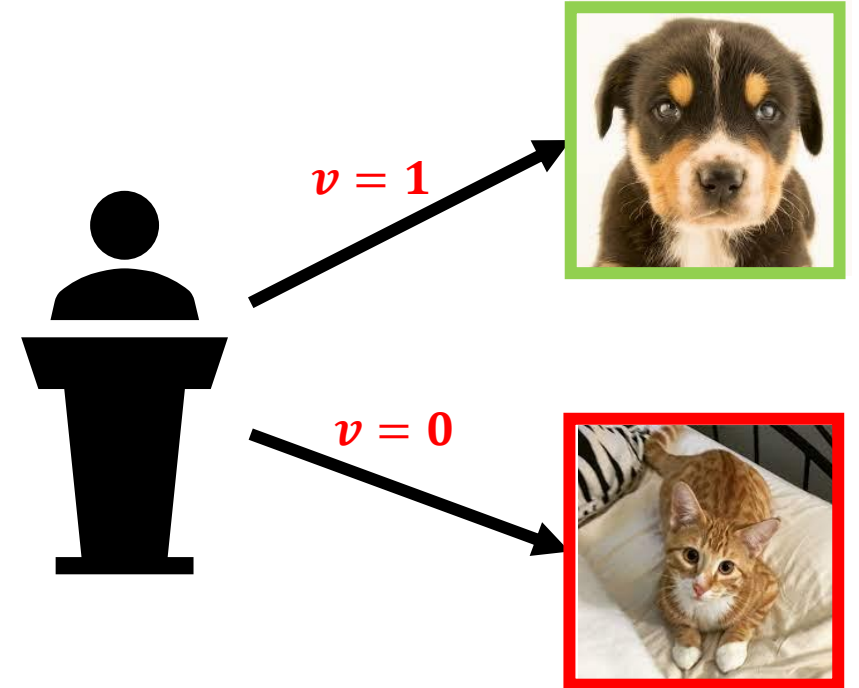


- n : number of training example
- $v \in [0,1]^n$: latent weight variable
- w : weight of **student** model
- $F(v, w)$: Loss function
- $g_s(x_i; w, y_i)$: prediction of DNN
- G : Curriculum
- γ : loss threshold

MentorMix Implementation

$$F(v, w) = \frac{1}{n} \sum_{i=1}^n v_i l(g_s(x_i; w), y_i) + \theta \|w\|_2^2 - G(v; \gamma)$$

Cross Entropy Loss



- n : number of training example
- $v \in [0,1]^n$: latent weight variable
- w : weight of **student** model
- $F(v, w)$: Loss function
- $g_s(x_i; w, y_i)$: prediction of DNN
- G : Curriculum
- γ : loss threshold

MentorMix Implementation

$$F(v, w)$$

$$= \frac{1}{n} \sum_{i=1}^n v_i l(g_s(x_i; w), y_i) + \theta \|w\|_2^2 - G(v; \gamma)$$

Curriculum

- n : number of training example
- $v \in [0,1]^n$: latent weight variable
- w : weight of **student** model
- $F(v, w)$: Loss function
- $g_s(x_i; w, y_i)$: prediction of DNN
- G : Curriculum
- γ : loss threshold

MentorMix Implementation

$$F(v, w)$$

$$= \frac{1}{n} \sum_{i=1}^n v_i l(g_s(x_i; w), y_i) + \theta \|w\|_2^2 - G(v; \gamma)$$

$$G(v; \gamma) = -\gamma v_i$$

- n : number of training example
- $v \in [0,1]^n$: latent weight variable
- w : weight of **student** model
- $F(v, w)$: Loss function
- $g_s(x_i; w, y_i)$: prediction of DNN
- G : Curriculum
- γ : loss threshold

MentorMix Implementation

$$F(v, w)$$

$$= \frac{1}{n} \sum_{i=1}^n v_i l(g_s(x_i; w), y_i) + \theta \|w\|_2^2 - \gamma v_i$$

$$= \frac{1}{n} \sum_{i=1}^n \boxed{v_i [l(g_s(x_i; w), y_i) - \gamma]} + \theta \|w\|_2^2$$

- n : number of training example
- $v \in [0,1]^n$: latent weight variable
- w : weight of **student** model
- $F(v, w)$: Loss function
- $g_s(x_i; w, y_i)$: prediction of DNN
- G : Curriculum
- γ : loss threshold

MentorMix Implementation

$$F(v, w)$$

$$= \frac{1}{n} \sum_{i=1}^n v_i l(g_s(x_i; w), y_i) + \theta \|w\|_2^2 - \gamma v_i$$

$$= \frac{1}{n} \sum_{i=1}^n v_i [l(g_s(x_i; w), y_i) - \gamma] + \theta \|w\|_2^2$$

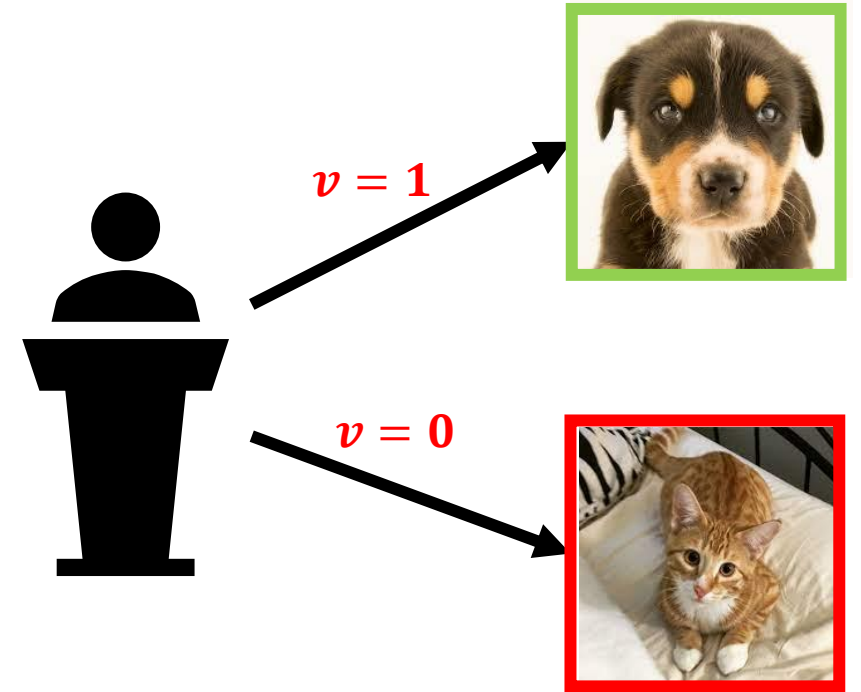
$$v_i = \begin{cases} 1 & \text{if } l(x_i, y_i) < \gamma \\ 0 & \text{if } l(x_i, y_i) > \gamma \end{cases}$$

- n : number of training example
- $v \in [0,1]^n$: latent weight variable
- w : weight of **student** model
- $F(v, w)$: Loss function
- $g_s(x_i; w, y_i)$: prediction of DNN
- G : Curriculum
- γ : loss threshold

MentorMix Implementation

$$\begin{aligned} F(v, w) &= \frac{1}{n} \sum_{i=1}^n v_i l(g_s(x_i; w), y_i) + \theta \|w\|_2^2 - \gamma v_i \\ &= \frac{1}{n} \sum_{i=1}^n v_i [l(g_s(x_i; w), y_i) - \gamma] + \theta \|w\|_2^2 \end{aligned}$$

$$v_i = \begin{cases} 1 & \text{if } l(x_i, y_i) < \gamma \\ 0 & \text{if } l(x_i, y_i) > \gamma \end{cases}$$



- n : number of training example
- $v \in [0,1]^n$: latent weight variable
- w : weight of **student** model
- $F(v, w)$: Loss function
- $g_s(x_i; w, y_i)$: prediction of DNN
- G : Curriculum
- γ : loss threshold

MentorMix Implementation

$$F(v, w) = \frac{1}{n} \sum_{i=1}^n v_i l(g_s(x_i; w), y_i) + \theta \|w\|_2^2 - G(v; \gamma)$$

$$\begin{aligned}\tilde{\mathbf{x}}_{ij} &= \lambda \mathbf{x}_i + (1 - \lambda) \mathbf{x}_j \\ \tilde{\mathbf{y}}_{ij} &= \lambda \mathbf{y}_i + (1 - \lambda) \mathbf{y}_j\end{aligned}$$

- $\lambda \sim \text{Beta}(\alpha, \alpha)$
- $\tilde{x}_{ij}$: mixed input
- $\tilde{y}_{ij}$: mixed output

MentorMix Implementation

$$F(v, w)$$

$$= \frac{1}{n} \sum_{i=1}^n v_i l(g_s(x_i; w), y_i) + \theta \|w\|_2^2 - G(v; \gamma)$$


$$\begin{aligned}\tilde{\mathbf{x}}_{ij} &= \lambda \mathbf{x}_i + (1 - \lambda) \mathbf{x}_j \\ \tilde{\mathbf{y}}_{ij} &= \lambda \mathbf{y}_i + (1 - \lambda) \mathbf{y}_j\end{aligned}$$

- $\lambda \sim \text{Beta}(\alpha, \alpha)$
- $\tilde{x}_{ij}$: mixed input
- $\tilde{y}_{ij}$: mixed output

MentorMix Implementation

$$F(v, w)$$

$$= \frac{1}{n^2} \sum_{i=1}^n \sum_{j=1}^n \mathbb{E}_{\lambda} [\tilde{v}_{ij} \ell(\tilde{\mathbf{x}}_{ij}, \tilde{\mathbf{y}}_{ij}) - \gamma \tilde{v}_{ij}]$$

- $\lambda \sim \text{Beta}(\alpha, \alpha)$
- $\tilde{\mathbf{x}}_{ij}$: mixed input
- $\tilde{\mathbf{y}}_{ij}$: mixed output
- $\tilde{v}_{ij} \in [0,1]^n$: latent weight variable

MentorMix Implementation

$$F(v, w)$$

$$= \frac{1}{n^2} \sum_{i=1}^n \sum_{j=1}^n \mathbb{E}_{\lambda} [\tilde{v}_{ij} \ell(\tilde{\mathbf{x}}_{ij}, \tilde{\mathbf{y}}_{ij}) - \gamma \tilde{v}_{ij}]$$

$$\tilde{v}_{ij} = \begin{cases} 1 & \text{if } l(\tilde{x}_{ij}, \tilde{y}_{ij}) < \gamma \\ 0 & \text{if } l(\tilde{x}_{ij}, \tilde{y}_{ij}) > \gamma \end{cases}$$

- $\lambda \sim \text{Beta}(\alpha, \alpha)$
- $\tilde{\mathbf{x}}_{ij}$: mixed input
- $\tilde{\mathbf{y}}_{ij}$: mixed output
- $\tilde{v}_{ij} \in [0,1]^n$: latent weight variable

MentorMix Implementation



**Too Many
Examples**

MentorMix Implementation

$v=0$



$v=1$



$v=1$



$v=0$



MentorMix Implementation

$v=0$



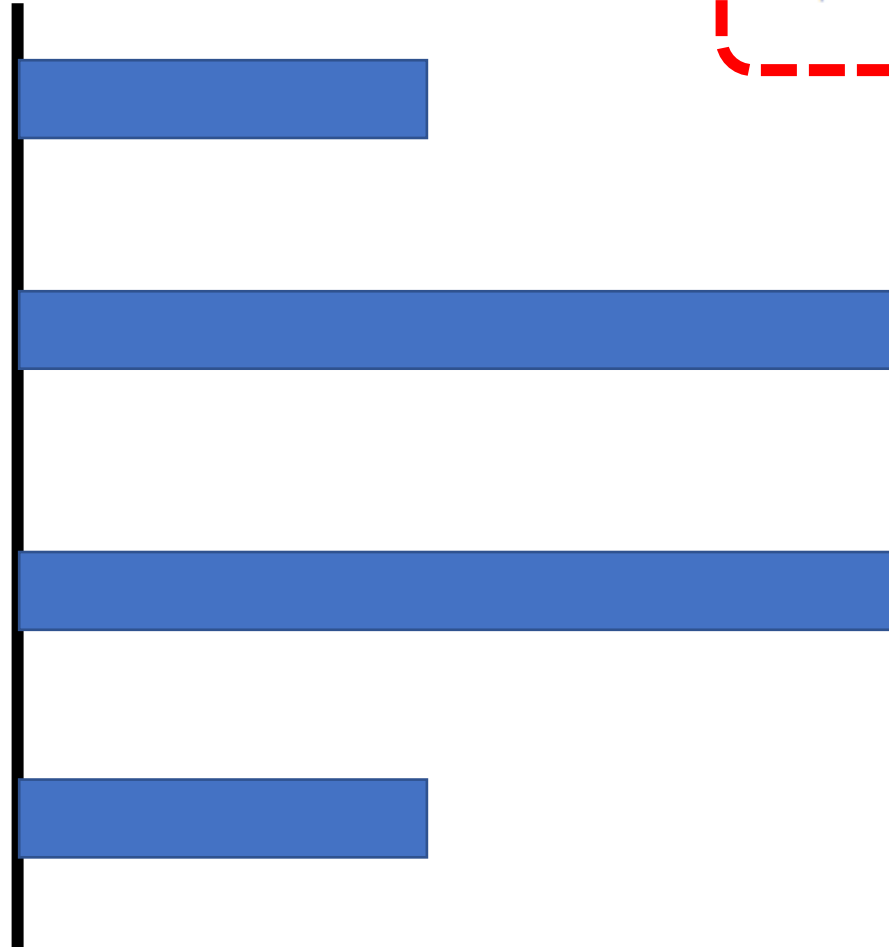
$v=1$



$v=1$

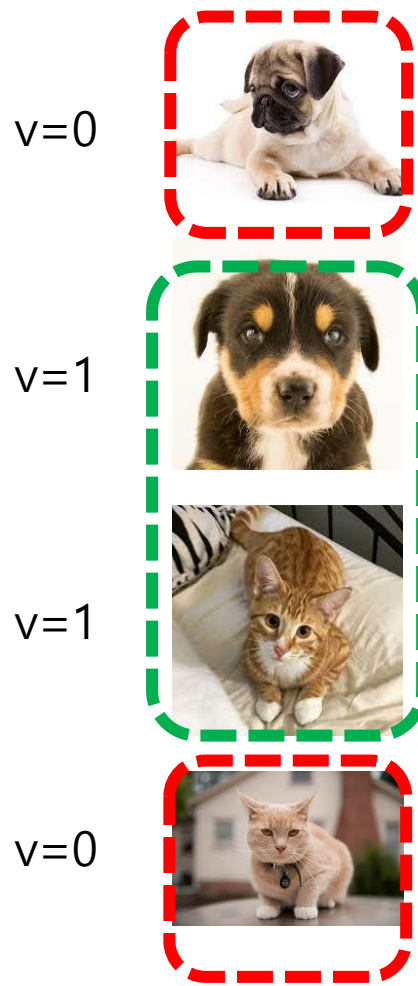


$v=0$



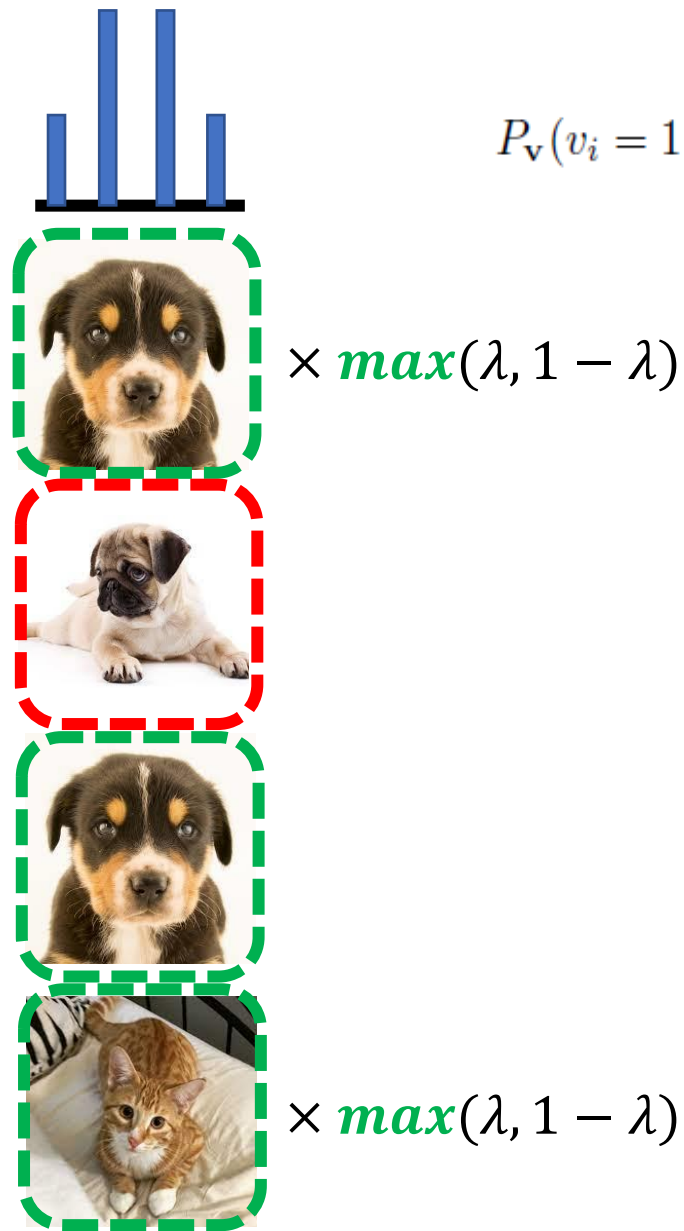
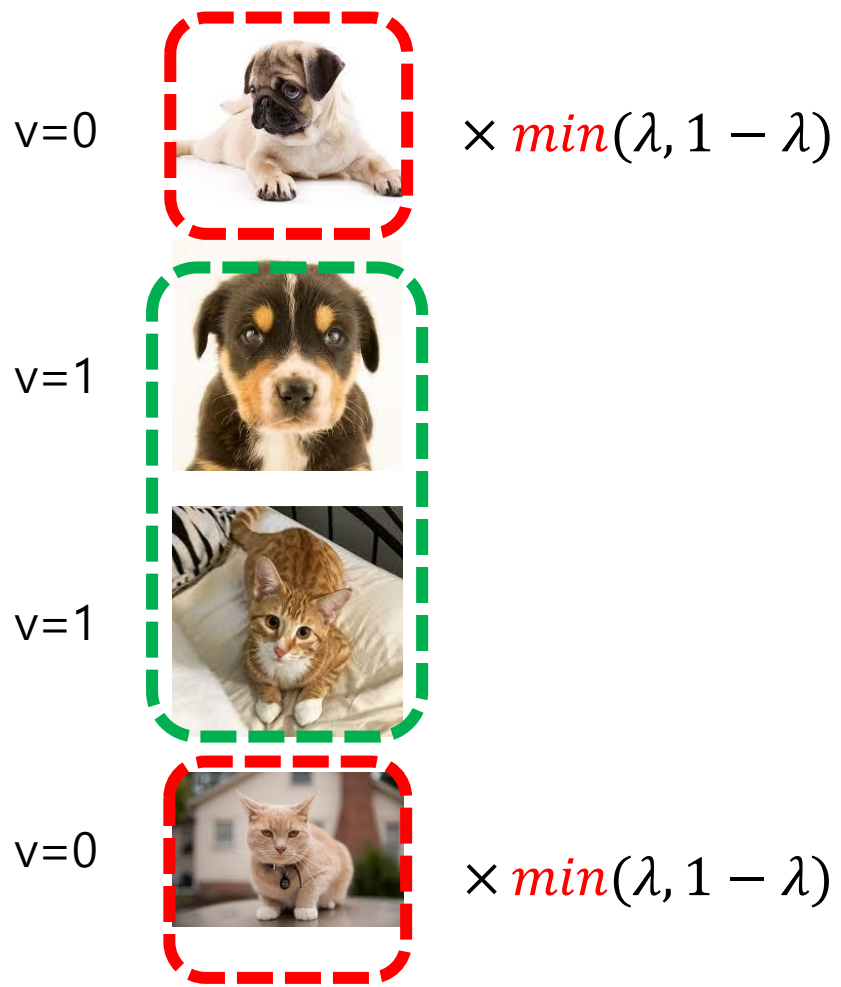
$$P_{\mathbf{v}}(v_i = 1 | \mathbf{x}_i, \mathbf{y}_i) = \frac{\exp(v_i^*/t)}{\sum_{j=1}^n \exp(v_j^*/t)}$$

MentorMix Implementation



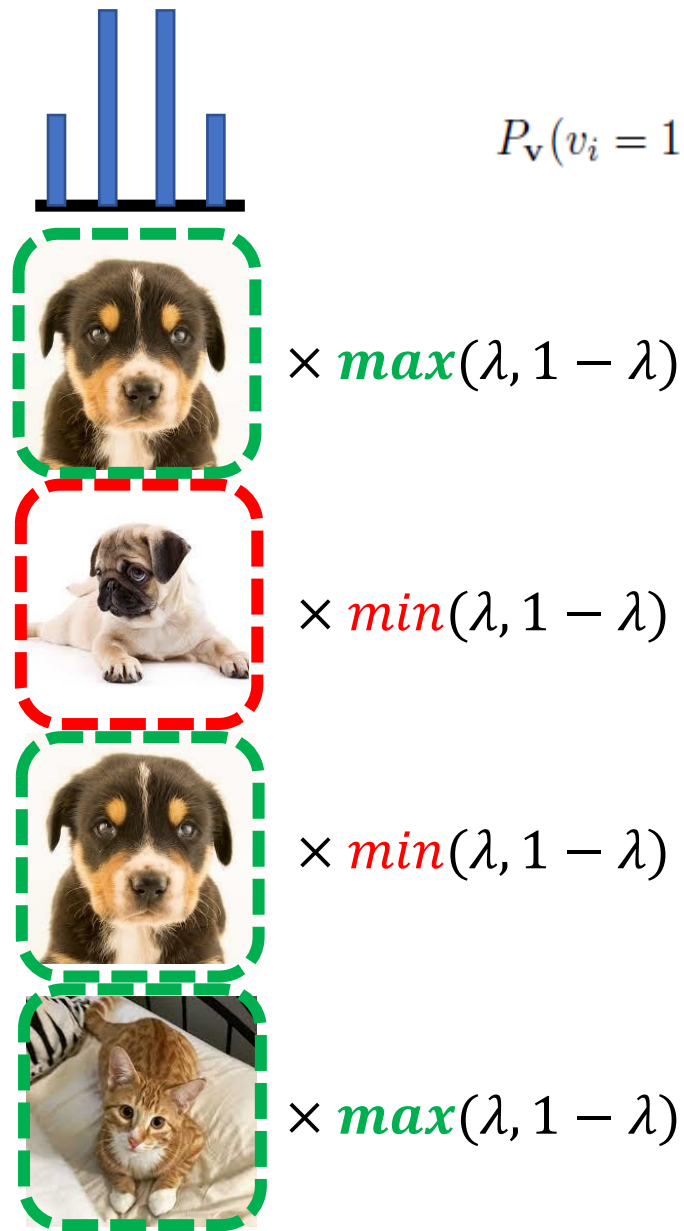
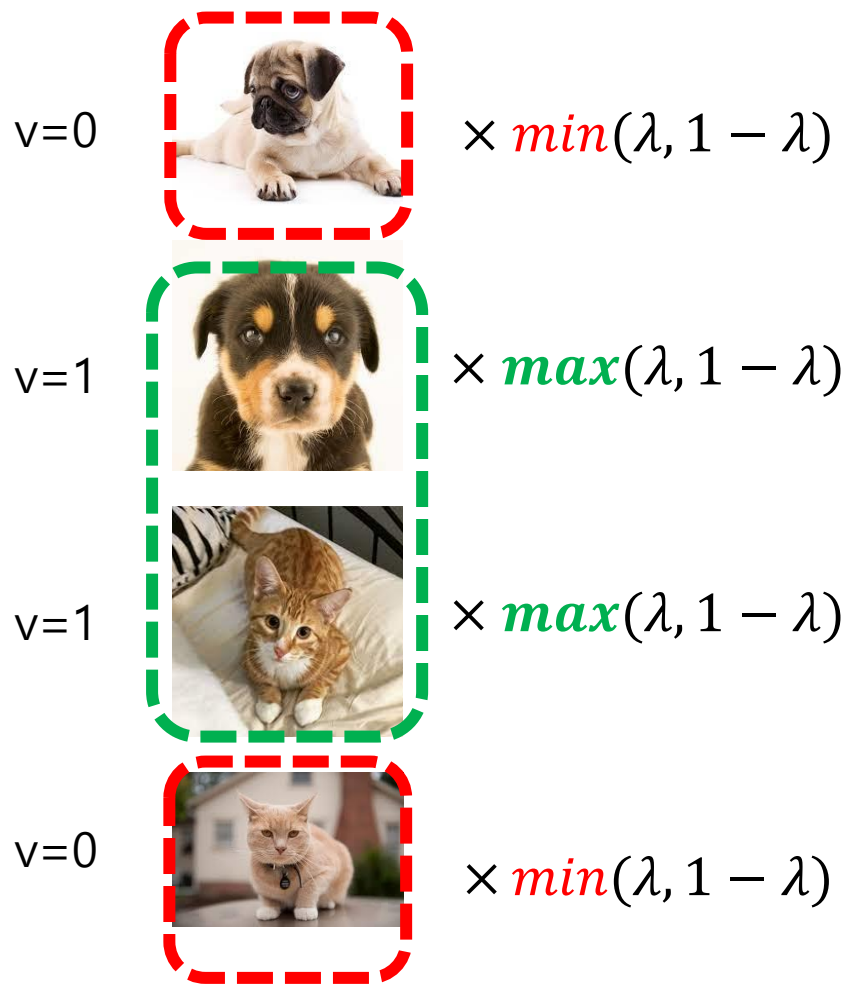
$$P_v(v_i = 1 | \mathbf{x}_i, y_i) = \frac{\exp(v_i^* / t)}{\sum_{j=1}^n \exp(v_j^* / t)}$$

MentorMix Implementation



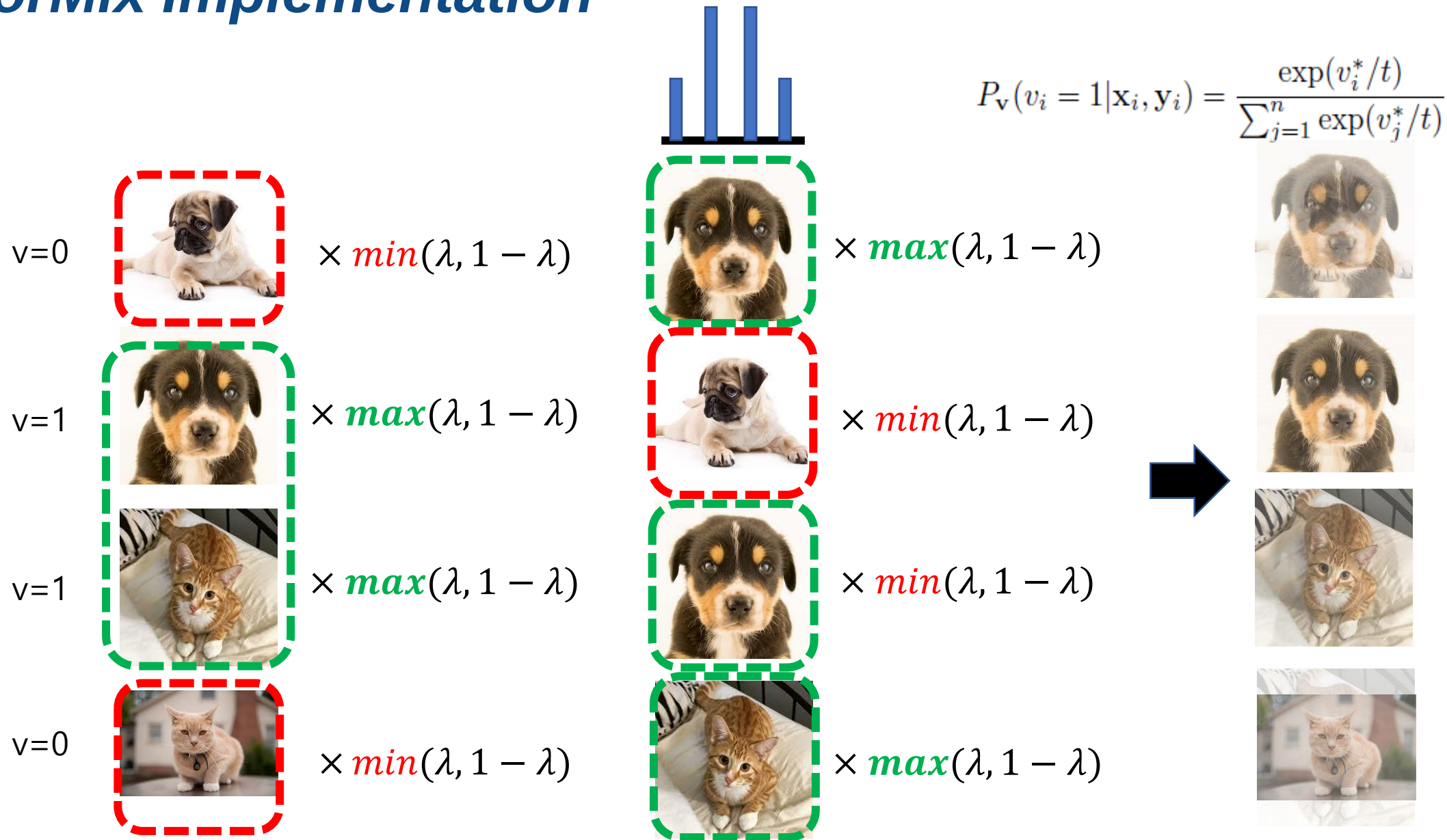
$$P_v(v_i = 1 | \mathbf{x}_i, \mathbf{y}_i) = \frac{\exp(v_i^* / t)}{\sum_{j=1}^n \exp(v_j^* / t)}$$

MentorMix Implementation



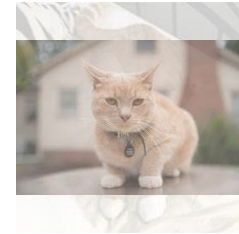
$$P_v(v_i = 1 | \mathbf{x}_i, \mathbf{y}_i) = \frac{\exp(v_i^* / t)}{\sum_{j=1}^n \exp(v_j^* / t)}$$

MentorMix Implementation



MentorMix Implementation

$$\begin{aligned} F(v, w) &= \sum_{(\mathbf{x}_i, \mathbf{y}_i) \sim \mathcal{D}} \sum_{(\mathbf{x}_j, \mathbf{y}_j) \sim \mathcal{D}} \mathbb{E}_{\lambda} [v_{ij} \ell(\tilde{\mathbf{x}}_{ij}, \tilde{\mathbf{y}}_{ij}) - \gamma v_{ij}] \\ &= \sum_{(\mathbf{x}_i, \mathbf{y}_i) \sim \mathcal{D}} \sum_{(\mathbf{x}_j, \mathbf{y}_j) \sim P_{\mathbf{v}}} \mathbb{E}_{\lambda} [\ell(\tilde{\mathbf{x}}_{ij}, \tilde{\mathbf{y}}_{ij})] - \gamma \end{aligned}$$



Algorithm 1 The proposed MentorMix method.

Input : mini-batch $\mathcal{D}_m$; two hyperparameters γ_p and α

Output : the loss of the mini-batch

- 1 For every $(\mathbf{x}_i, y_i)$ in $\mathcal{D}_m$ compute $\ell(\mathbf{x}_i, y_i)$
 - 2 Set $\ell_p(\mathcal{D}_m)$ to be the γ_p -th percentile of the loss $\{\ell(\mathbf{x}_i, y_i)\}$.
 - 3 $\gamma \leftarrow \text{EMA}(\ell_p(\mathcal{D}_m))$ // update the moving average
 - 4 $v_i^* \leftarrow \text{MentorNet}(\ell(\mathbf{x}_i, y_i), \gamma)$ // MentorNet weight
 - 5 Compute $P_{\mathbf{v}} = \text{softmax}(\mathbf{v}^*)$, where $\mathbf{v}^* = [v_1^*, \dots, v_{|\mathcal{D}_m|}^*]$
 - 6 Stop gradient
 - 7 **foreach** $(\mathbf{x}_i, y_i)$ **do**
 - 8 Draw a sample $(\mathbf{x}_j, y_j)$ with replacement from $P_{\mathbf{v}}$
 - 9 $\lambda \leftarrow \text{Beta}(\alpha, \alpha)$
 - 10 $\lambda \leftarrow v_i^* \max(\lambda, 1 - \lambda) + (1 - v_i^*) \min(\lambda, 1 - \lambda)$
 - 11 $\tilde{\mathbf{x}}_{ij} \leftarrow \lambda \mathbf{x}_i + (1 - \lambda) \mathbf{x}_j$
 - 12 $\tilde{y}_{ij} \leftarrow \lambda y_i + (1 - \lambda) y_j$
 - 13 Compute $\ell_i = \ell(\tilde{\mathbf{x}}_{ij}, \tilde{y}_{ij})$
 - 14 Weight ℓ_i using a separate MentorNet // optional
 - 15 **end**
 - 16 **return** $(1/|\mathcal{D}_m|) \sum_{i=1}^{|\mathcal{D}_m|} \ell_i$
-

Algorithm 1 The proposed MentorMix method.

Input : mini-batch $\mathcal{D}_m$; two hyperparameters γ_p and α

Output: the loss of the mini-batch

```
1 For every  $(\mathbf{x}_i, y_i)$  in  $\mathcal{D}_m$  compute  $\ell(\mathbf{x}_i, y_i)$ 
2 Set  $\ell_p(\mathcal{D}_m)$  to be the  $\gamma_p$ -th percentile of the loss  $\{\ell(\mathbf{x}_i, y_i)\}$ .
3  $\gamma \leftarrow \text{EMA}(\ell_p(\mathcal{D}_m))$  // update the moving average
4  $v_i^* \leftarrow \text{MentorNet}(\ell(\mathbf{x}_i, y_i), \gamma)$  // MentorNet weight
5 Compute  $P_v = \text{softmax}(\mathbf{v}^*)$ , where  $\mathbf{v}^* = [v_1^*, \dots, v_{|\mathcal{D}_m|}^*]$ 
6 Stop gradient
7 foreach  $(\mathbf{x}_i, y_i)$  do
8   Draw a sample  $(\mathbf{x}_j, y_j)$  with replacement from  $P_v$ 
9    $\lambda \leftarrow \text{Beta}(\alpha, \alpha)$ 
10   $\lambda \leftarrow v_i^* \max(\lambda, 1 - \lambda) + (1 - v_i^*) \min(\lambda, 1 - \lambda)$ 
11   $\tilde{\mathbf{x}}_{ij} \leftarrow \lambda \mathbf{x}_i + (1 - \lambda) \mathbf{x}_j$ 
12   $\tilde{\mathbf{y}}_{ij} \leftarrow \lambda y_i + (1 - \lambda) y_j$ 
13  Compute  $\ell_i = \ell(\tilde{\mathbf{x}}_{ij}, \tilde{\mathbf{y}}_{ij})$ 
14  Weight  $\ell_i$  using a separate MentorNet // optional
15 end
16 return  $(1/|\mathcal{D}_m|) \sum_{i=1}^{|\mathcal{D}_m|} \ell_i$ 
```

- x_i : i 'th input
- y_i : i 'th label
- $\mathcal{D}_m$: mini batch
- α, γ_p : hyper parameter
- $v \in [0,1]^n$: latent weight variable
- P_v : Sampling Distribution
- $\lambda \sim \text{Beta}(\alpha, \alpha)$
- $\tilde{\mathbf{x}}_{ij}$: mixed input
- $\tilde{\mathbf{y}}_{ij}$: mixed output

Algorithm 1 The proposed MentorMix method.

Input : mini-batch $\mathcal{D}_m$; two hyperparameters γ_p and α

Output : the loss of the mini-batch

```
1 For every  $(\mathbf{x}_i, y_i)$  in  $\mathcal{D}_m$  compute  $\ell(\mathbf{x}_i, y_i)$ 
2 Set  $\ell_p(\mathcal{D}_m)$  to be the  $\gamma_p$ -th percentile of the loss  $\{\ell(\mathbf{x}_i, y_i)\}$ .
3  $\gamma \leftarrow \text{EMA}(\ell_p(\mathcal{D}_m))$  // update the moving average
4  $v_i^* \leftarrow \text{MentorNet}(\ell(\mathbf{x}_i, y_i), \gamma)$  // MentorNet weight
5 Compute  $P_v = \text{softmax}(\mathbf{v}^*)$ , where  $\mathbf{v}^* = [v_1^*, \dots, v_{|\mathcal{D}_m|}^*]$ 
6 Stop gradient
7 foreach  $(\mathbf{x}_i, y_i)$  do
8   Draw a sample  $(\mathbf{x}_j, y_j)$  with replacement from  $P_v$ 
9    $\lambda \leftarrow \text{Beta}(\alpha, \alpha)$ 
10   $\lambda \leftarrow v_i^* \max(\lambda, 1 - \lambda) + (1 - v_i^*) \min(\lambda, 1 - \lambda)$ 
11   $\tilde{\mathbf{x}}_{ij} \leftarrow \lambda \mathbf{x}_i + (1 - \lambda) \mathbf{x}_j$ 
12   $\tilde{\mathbf{y}}_{ij} \leftarrow \lambda y_i + (1 - \lambda) y_j$ 
13  Compute  $\ell_i = \ell(\tilde{\mathbf{x}}_{ij}, \tilde{\mathbf{y}}_{ij})$ 
14  Weight  $\ell_i$  using a separate MentorNet // optional
15 end
16 return  $(1/|\mathcal{D}_m|) \sum_{i=1}^{|\mathcal{D}_m|} \ell_i$ 
```

* $\gamma_p \in \{90\%, 80\%, 70\%\}$

- x_i : i 'th input
- y_i : i 'th label
- D_m : mini batch
- α, γ_p : hyper parameter
- $v \in [0,1]^n$: latent weight variable
- P_v : Sampling Distribution
- $\lambda \sim \text{Beta}(\alpha, \alpha)$
- $\tilde{x}_{ij}$: mixed input
- $\tilde{y}_{ij}$: mixed output

Algorithm 1 The proposed MentorMix method.

Input : mini-batch $\mathcal{D}_m$; two hyperparameters γ_p and α

Output : the loss of the mini-batch

```
1 For every  $(\mathbf{x}_i, y_i)$  in  $\mathcal{D}_m$  compute  $\ell(\mathbf{x}_i, y_i)$ 
2 Set  $\ell_p(\mathcal{D}_m)$  to be the  $\gamma_p$ -th percentile of the loss  $\{\ell(\mathbf{x}_i, y_i)\}$ .
3  $\gamma \leftarrow \text{EMA}(\ell_p(\mathcal{D}_m))$  // update the moving average
4  $v_i^* \leftarrow \text{MentorNet}(\ell(\mathbf{x}_i, y_i), \gamma)$  // MentorNet weight
5 Compute  $P_v = \text{softmax}(\mathbf{v}^*)$ , where  $\mathbf{v}^* = [v_1^*, \dots, v_{|\mathcal{D}_m|}^*]$ 
6 Stop gradient
7 foreach  $(\mathbf{x}_i, y_i)$  do
8   Draw a sample  $(\mathbf{x}_j, y_j)$  with replacement from  $P_v$ 
9    $\lambda \leftarrow \text{Beta}(\alpha, \alpha)$ 
10   $\lambda \leftarrow v_i^* \max(\lambda, 1 - \lambda) + (1 - v_i^*) \min(\lambda, 1 - \lambda)$ 
11   $\tilde{\mathbf{x}}_{ij} \leftarrow \lambda \mathbf{x}_i + (1 - \lambda) \mathbf{x}_j$ 
12   $\tilde{\mathbf{y}}_{ij} \leftarrow \lambda y_i + (1 - \lambda) y_j$ 
13  Compute  $\ell_i = \ell(\tilde{\mathbf{x}}_{ij}, \tilde{\mathbf{y}}_{ij})$ 
14  Weight  $\ell_i$  using a separate MentorNet // optional
15 end
16 return  $(1/|\mathcal{D}_m|) \sum_{i=1}^{|\mathcal{D}_m|} \ell_i$ 
```

* $\gamma_p \in \{90\%, 80\%, 70\%\}$

* EMA weight = 0.999

- x_i : i 'th input
- y_i : i 'th label
- D_m : mini batch
- α, γ_p : hyper parameter
- $v \in [0,1]^n$: latent weight variable
- P_v : Sampling Distribution
- $\lambda \sim \text{Beta}(\alpha, \alpha)$
- $\tilde{x}_{ij}$: mixed input
- $\tilde{y}_{ij}$: mixed output

Algorithm 1 The proposed MentorMix method.

Input : mini-batch $\mathcal{D}_m$; two hyperparameters γ_p and α

Output : the loss of the mini-batch

```
1 For every  $(\mathbf{x}_i, y_i)$  in  $\mathcal{D}_m$  compute  $\ell(\mathbf{x}_i, y_i)$ 
2 Set  $\ell_p(\mathcal{D}_m)$  to be the  $\gamma_p$ -th percentile of the loss  $\{\ell(\mathbf{x}_i, y_i)\}$ .
3  $\gamma \leftarrow \text{EMA}(\ell_p(\mathcal{D}_m))$  // update the moving average
4  $v_i^* \leftarrow \text{MentorNet}(\ell(\mathbf{x}_i, y_i), \gamma)$  // MentorNet weight
5 Compute  $P_v = \text{softmax}(\mathbf{v}^*)$ , where  $\mathbf{v}^* = [v_1^*, \dots, v_{|\mathcal{D}_m|}^*]$ 
6 Stop gradient
7 foreach  $(\mathbf{x}_i, y_i)$  do
8   Draw a sample  $(\mathbf{x}_j, y_j)$  with replacement from  $P_v$ 
9    $\lambda \leftarrow \text{Beta}(\alpha, \alpha)$ 
10   $\lambda \leftarrow v_i^* \max(\lambda, 1 - \lambda) + (1 - v_i^*) \min(\lambda, 1 - \lambda)$ 
11   $\tilde{\mathbf{x}}_{ij} \leftarrow \lambda \mathbf{x}_i + (1 - \lambda) \mathbf{x}_j$ 
12   $\tilde{\mathbf{y}}_{ij} \leftarrow \lambda y_i + (1 - \lambda) y_j$ 
13  Compute  $\ell_i = \ell(\tilde{\mathbf{x}}_{ij}, \tilde{\mathbf{y}}_{ij})$ 
14  Weight  $\ell_i$  using a separate MentorNet // optional
15 end
16 return  $(1/|\mathcal{D}_m|) \sum_{i=1}^{|\mathcal{D}_m|} \ell_i$ 
```

- x_i : i 'th input
- y_i : i 'th label
- $\mathcal{D}_m$: mini batch
- α, γ_p : hyper parameter
- $v \in [0,1]^n$: latent weight variable
- P_v : Sampling Distribution
- $\lambda \sim \text{Beta}(\alpha, \alpha)$
- $\tilde{\mathbf{x}}_{ij}$: mixed input
- $\tilde{\mathbf{y}}_{ij}$: mixed output

Algorithm 1 The proposed MentorMix method.

Input : mini-batch $\mathcal{D}_m$; two hyperparameters γ_p and α

Output : the loss of the mini-batch

```
1 For every  $(\mathbf{x}_i, y_i)$  in  $\mathcal{D}_m$  compute  $\ell(\mathbf{x}_i, y_i)$ 
2 Set  $\ell_p(\mathcal{D}_m)$  to be the  $\gamma_p$ -th percentile of the loss  $\{\ell(\mathbf{x}_i, y_i)\}$ .
3  $\gamma \leftarrow \text{EMA}(\ell_p(\mathcal{D}_m))$  // update the moving average
4  $v_i^* \leftarrow \text{MentorNet}(\ell(\mathbf{x}_i, y_i), \gamma)$  // MentorNet weight
5 Compute  $P_v = \text{softmax}(\mathbf{v}^*)$ , where  $\mathbf{v}^* = [v_1^*, \dots, v_{|\mathcal{D}_m|}^*]$ 
6 Stop gradient
7 foreach  $(\mathbf{x}_i, y_i)$  do
8   Draw a sample  $(\mathbf{x}_j, y_j)$  with replacement from  $P_v$ 
9    $\lambda \leftarrow \text{Beta}(\alpha, \alpha)$ 
10   $\lambda \leftarrow v_i^* \max(\lambda, 1 - \lambda) + (1 - v_i^*) \min(\lambda, 1 - \lambda)$ 
11   $\tilde{\mathbf{x}}_{ij} \leftarrow \lambda \mathbf{x}_i + (1 - \lambda) \mathbf{x}_j$ 
12   $\tilde{\mathbf{y}}_{ij} \leftarrow \lambda y_i + (1 - \lambda) y_j$ 
13  Compute  $\ell_i = \ell(\tilde{\mathbf{x}}_{ij}, \tilde{\mathbf{y}}_{ij})$ 
14  Weight  $\ell_i$  using a separate MentorNet // optional
15 end
16 return  $(1/|\mathcal{D}_m|) \sum_{i=1}^{|\mathcal{D}_m|} \ell_i$ 
```

- x_i : i 'th input
- y_i : i 'th label
- $\mathcal{D}_m$: mini batch
- α, γ_p : hyper parameter
- $v \in [0,1]^n$: latent weight variable
- P_v : Sampling Distribution
- $\lambda \sim \text{Beta}(\alpha, \alpha)$
- $\tilde{\mathbf{x}}_{ij}$: mixed input
- $\tilde{\mathbf{y}}_{ij}$: mixed output

Algorithm 1 The proposed MentorMix method.

Input : mini-batch $\mathcal{D}_m$; two hyperparameters γ_p and α

Output : the loss of the mini-batch

```
1 For every  $(\mathbf{x}_i, y_i)$  in  $\mathcal{D}_m$  compute  $\ell(\mathbf{x}_i, y_i)$ 
2 Set  $\ell_p(\mathcal{D}_m)$  to be the  $\gamma_p$ -th percentile of the loss  $\{\ell(\mathbf{x}_i, y_i)\}$ .
3  $\gamma \leftarrow \text{EMA}(\ell_p(\mathcal{D}_m))$  // update the moving average
4  $v_i^* \leftarrow \text{MentorNet}(\ell(\mathbf{x}_i, y_i), \gamma)$  // MentorNet weight
5 Compute  $P_v = \text{softmax}(\mathbf{v}^*)$ , where  $\mathbf{v}^* = [v_1^*, \dots, v_{|\mathcal{D}_m|}^*]$ 
6 Stop gradient
7 foreach  $(\mathbf{x}_i, y_i)$  do
8   Draw a sample  $(\mathbf{x}_j, y_j)$  with replacement from  $P_v$ 
9    $\lambda \leftarrow \text{Beta}(\alpha, \alpha)$ 
10   $\lambda \leftarrow v_i^* \max(\lambda, 1 - \lambda) + (1 - v_i^*) \min(\lambda, 1 - \lambda)$ 
11   $\tilde{\mathbf{x}}_{ij} \leftarrow \lambda \mathbf{x}_i + (1 - \lambda) \mathbf{x}_j$ 
12   $\tilde{\mathbf{y}}_{ij} \leftarrow \lambda y_i + (1 - \lambda) y_j$ 
13  Compute  $\ell_i = \ell(\tilde{\mathbf{x}}_{ij}, \tilde{\mathbf{y}}_{ij})$ 
14  Weight  $\ell_i$  using a separate MentorNet // optional
15 end
16 return  $(1/|\mathcal{D}_m|) \sum_{i=1}^{|\mathcal{D}_m|} \ell_i$ 
```

- x_i : i 'th input
- y_i : i 'th label
- $\mathcal{D}_m$: mini batch
- α, γ_p : hyper parameter
- $v \in [0,1]^n$: latent weight variable
- P_v : Sampling Distribution
- $\lambda \sim \text{Beta}(\alpha, \alpha)$
- $\tilde{\mathbf{x}}_{ij}$: mixed input
- $\tilde{\mathbf{y}}_{ij}$: mixed output

Algorithm 1 The proposed MentorMix method.

Input : mini-batch $\mathcal{D}_m$; two hyperparameters γ_p and α

Output : the loss of the mini-batch

```
1 For every  $(\mathbf{x}_i, y_i)$  in  $\mathcal{D}_m$  compute  $\ell(\mathbf{x}_i, y_i)$ 
2 Set  $\ell_p(\mathcal{D}_m)$  to be the  $\gamma_p$ -th percentile of the loss  $\{\ell(\mathbf{x}_i, y_i)\}$ .
3  $\gamma \leftarrow \text{EMA}(\ell_p(\mathcal{D}_m))$  // update the moving average
4  $v_i^* \leftarrow \text{MentorNet}(\ell(\mathbf{x}_i, y_i), \gamma)$  // MentorNet weight
5 Compute  $P_v = \text{softmax}(\mathbf{v}^*)$ , where  $\mathbf{v}^* = [v_1^*, \dots, v_{|\mathcal{D}_m|}^*]$ 
6 Stop gradient
7 foreach  $(\mathbf{x}_i, y_i)$  do
8   Draw a sample  $(\mathbf{x}_j, y_j)$  with replacement from  $P_v$ 
9    $\lambda \leftarrow \text{Beta}(\alpha, \alpha)$ 
10   $\lambda \leftarrow v_i^* \max(\lambda, 1 - \lambda) + (1 - v_i^*) \min(\lambda, 1 - \lambda)$ 
11   $\tilde{\mathbf{x}}_{ij} \leftarrow \lambda \mathbf{x}_i + (1 - \lambda) \mathbf{x}_j$ 
12   $\tilde{y}_{ij} \leftarrow \lambda y_i + (1 - \lambda) y_j$ 
13  Compute  $\ell_i = \ell(\tilde{\mathbf{x}}_{ij}, \tilde{y}_{ij})$ 
14  Weight  $\ell_i$  using a separate MentorNet // optional
15 end
16 return  $(1/|\mathcal{D}_m|) \sum_{i=1}^{|\mathcal{D}_m|} \ell_i$ 
```

- x_i : i 'th input
- y_i : i 'th label
- $\mathcal{D}_m$: mini batch
- α, γ_p : hyper parameter
- $v \in [0,1]^n$: latent weight variable
- P_v : Sampling Distribution
- $\lambda \sim \text{Beta}(\alpha, \alpha)$
- $\tilde{x}_{ij}$: mixed input
- $\tilde{y}_{ij}$: mixed output

Algorithm 1 The proposed MentorMix method.

Input : mini-batch $\mathcal{D}_m$; two hyperparameters γ_p and α

Output : the loss of the mini-batch

```
1 For every  $(\mathbf{x}_i, y_i)$  in  $\mathcal{D}_m$  compute  $\ell(\mathbf{x}_i, y_i)$ 
2 Set  $\ell_p(\mathcal{D}_m)$  to be the  $\gamma_p$ -th percentile of the loss  $\{\ell(\mathbf{x}_i, y_i)\}$ .
3  $\gamma \leftarrow \text{EMA}(\ell_p(\mathcal{D}_m))$  // update the moving average
4  $v_i^* \leftarrow \text{MentorNet}(\ell(\mathbf{x}_i, y_i), \gamma)$  // MentorNet weight
5 Compute  $P_v = \text{softmax}(\mathbf{v}^*)$ , where  $\mathbf{v}^* = [v_1^*, \dots, v_{|\mathcal{D}_m|}^*]$ 
6 Stop gradient
7 foreach  $(\mathbf{x}_i, y_i)$  do
8   Draw a sample  $(\mathbf{x}_j, y_j)$  with replacement from  $P_v$ 
9    $\lambda \leftarrow \text{Beta}(\alpha, \alpha)$ 
10   $\lambda \leftarrow v_i^* \max(\lambda, 1 - \lambda) + (1 - v_i^*) \min(\lambda, 1 - \lambda)$ 
11   $\tilde{\mathbf{x}}_{ij} \leftarrow \lambda \mathbf{x}_i + (1 - \lambda) \mathbf{x}_j$ 
12   $\tilde{y}_{ij} \leftarrow \lambda y_i + (1 - \lambda) y_j$ 
13  Compute  $\ell_i = \ell(\tilde{\mathbf{x}}_{ij}, \tilde{y}_{ij})$ 
14  Weight  $\ell_i$  using a separate MentorNet // optional
15 end
16 return  $(1/|\mathcal{D}_m|) \sum_{i=1}^{|\mathcal{D}_m|} \ell_i$ 
```

- x_i : i 'th input
- y_i : i 'th label
- $\mathcal{D}_m$: mini batch
- α, γ_p : hyper parameter
- $v \in [0,1]^n$: latent weight variable
- P_v : Sampling Distribution
- $\lambda \sim \text{Beta}(\alpha, \alpha)$
- $\tilde{x}_{ij}$: mixed input
- $\tilde{y}_{ij}$: mixed output

4. Experiments

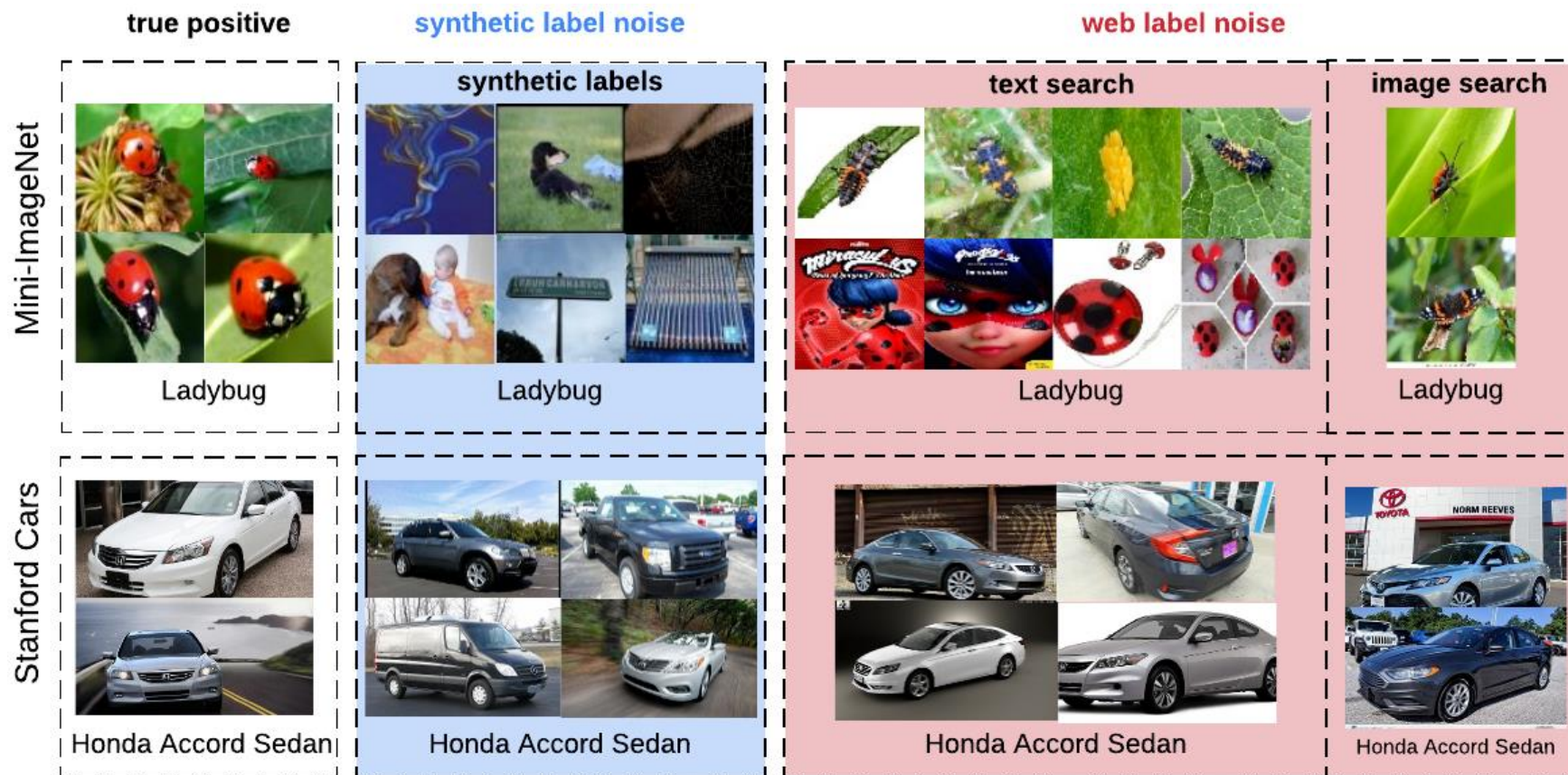


Table 1. Overview of our datasets of controlled red (web) label noise. Blue (synthetic) label noise is also included for comparison.

Dataset	#Class	Noise Source	Train Size	Val Size	Controlled Noise Levels (%)
Red Mini-ImageNet	100	image search label	50,000	5,000	0, 5, 10, 15, 20, 30, 40, 50, 60, 80
Blue Mini-ImageNet		symmetric label flipping	60,000		0, 5, 10, 15, 20, 30, 40, 50, 60, 80
Red Stanford Cars	196	image search label	8,144	8,041	0, 5, 10, 15, 20, 30, 40, 50, 60, 80
Blue Stanford Cars		symmetric label flipping	8,144		0, 5, 10, 15, 20, 30, 40, 50, 60, 80

Experiments Setting

- All Methods trained on **BLUE/RED** Noise **Mini-ImageNet / Stanford** under 10 noise levels(p) [0,5,10,15,20,30,40,50,60,80]
- Train from **Scratch / Fine-Tuning**
- **Inception-ResNet-v2**(upsample Mini-ImageNet dataset from 84x84 to 299x299)
- **Grid Search** Hyperparameter $\alpha \in \{0.4, 1, 2\}$, $\gamma_p \in \{90\%, 80\%, 70\%\}$

Table 2. Peak accuracy (%) of the best trial of each method averaged across 10 noise levels. – denotes the method failed to train.

Method	Mini-ImageNet				Stanford Cars			
	Fine-tuned		Trained from scratch		Fine-tuned		Trained from scratch	
	Blue	Red	Blue	Red	Blue	Red	Blue	Red
Vanilla	82.3 ± 1.9	81.6 ± 1.9	58.3 ± 10.3	64.9 ± 5.2	70.0 ± 16.8	82.4 ± 6.9	53.8 ± 24.4	77.7 ± 10.4
WeightDecay	81.9 ± 1.8	81.5 ± 1.8	—	—	72.2 ± 17.5	84.3 ± 6.6	—	—
Dropout	82.8 ± 1.3	81.8 ± 1.8	59.3 ± 9.5	65.7 ± 5.0	71.7 ± 16.9	83.8 ± 6.6	62.8 ± 23.5	84.1 ± 6.7
S-Model	82.3 ± 1.8	82.0 ± 1.9	58.7 ± 10.2	64.6 ± 5.1	69.7 ± 16.8	82.4 ± 7.1	53.9 ± 23.5	77.6 ± 10.2
Bootstrap	83.1 ± 1.6	82.7 ± 1.8	60.1 ± 9.7	65.5 ± 4.9	71.7 ± 16.9	82.8 ± 6.7	55.6 ± 23.9	78.9 ± 9.6
Mixup	81.7 ± 1.8	82.4 ± 1.7	60.7 ± 9.8	66.0 ± 4.9	73.1 ± 16.6	85.0 ± 6.2	64.2 ± 21.6	82.5 ± 8.0
MentorNet	82.9 ± 1.7	82.4 ± 1.7	61.8 ± 10.3	65.1 ± 5.0	75.9 ± 16.8	82.6 ± 6.6	56.8 ± 23.1	78.9 ± 8.9
Our MentorMix	84.2 ± 0.7	83.3 ± 1.9	70.9 ± 3.4	67.0 ± 5.0	78.2 ± 16.2	86.9 ± 5.5	67.7 ± 23.0	83.6 ± 7.5

Table 3. Comparison with the state-of-the-art in terms of the validation accuracy on CIFAR-100 (top) and CIFAR-10 (bottom).

Data	Method	Noise level (%)			
		20	40	60	80
CIFAR100	Arazo et al. (2019)	73.7	70.1	59.5	39.5
	Zhang & Sabuncu (2018)	67.6	62.6	54.0	29.6
	MentorNet (2018)	73.5	68.5	61.2	35.5
	Mixup (2018)	73.9	66.8	58.8	40.1
	Huang et al. (2019)	74.1	69.2	39.4	-
	Ours (MentorMix)	78.6	71.3	64.6	41.2
CIFAR10	Arazo et al. (2019)	94.0	92.8	90.3	74.1
	Zhang & Sabuncu (2018)	89.7	87.6	82.7	67.9
	Lee et al. (2019)	87.1	81.8	75.4	-
	Chen et al. (2019)	89.7	-	-	52.3
	Huang et al. (2019)	92.6	90.3	43.4	-
	MentorNet (2018)	92.0	91.2	74.2	60.0
	Mixup (2018)	94.0	91.5	86.8	76.9
	Ours (MentorMix) [†]	95.6	94.2	91.3	81.0

Understanding DNNs trained on noisy labels

Understanding DNNs trained on noisy labels

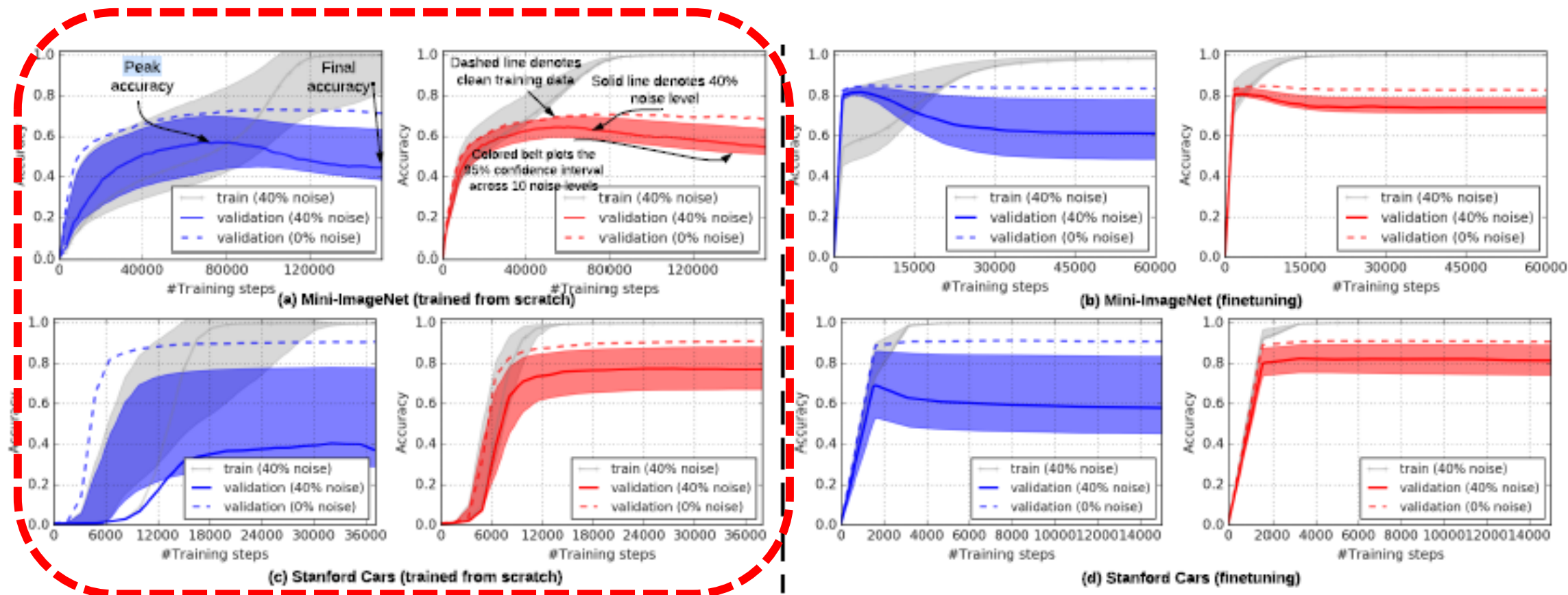


Figure 2. DNNs trained on synthetic (blue) and web label noise (red) on Mini-ImageNet (top) and Stanford Cars (bottom).

Understanding DNNs trained on noisy labels

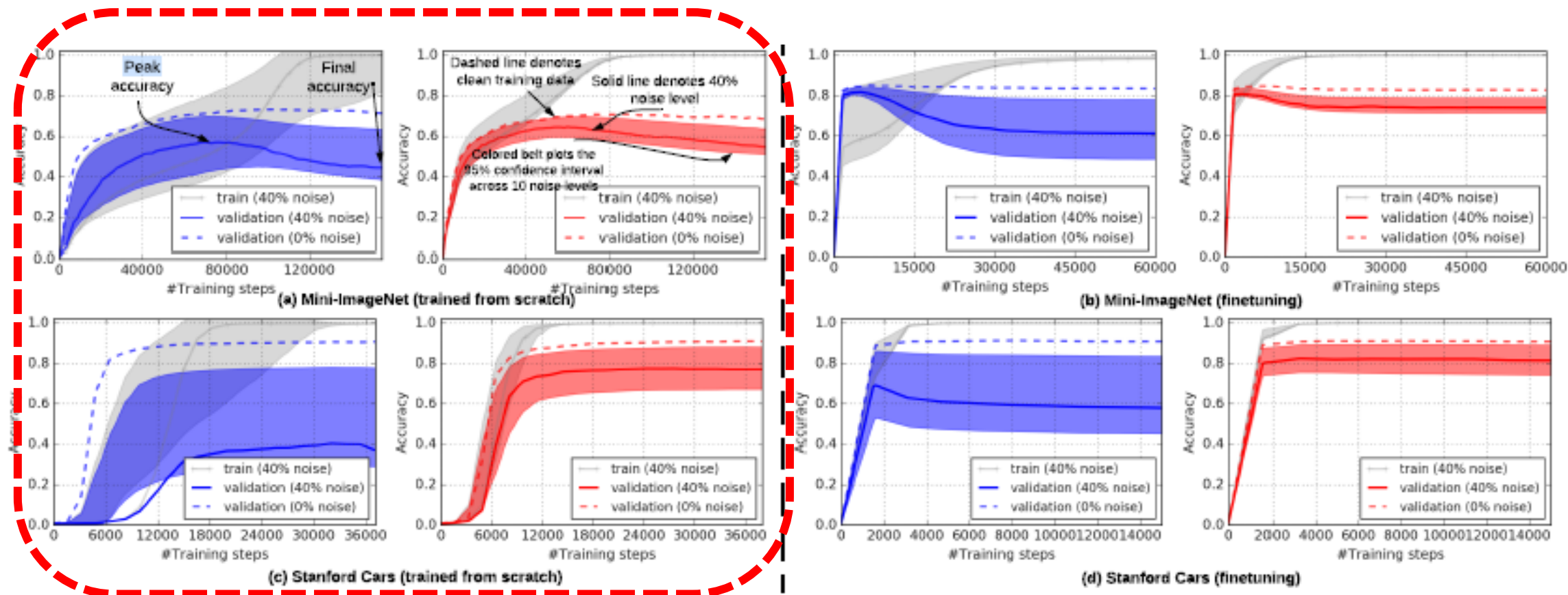


Figure 2. DNNs trained on synthetic (blue) and web label noise (red) on Mini-ImageNet (top) and Stanford Cars (bottom).

Understanding DNNs trained on noisy labels

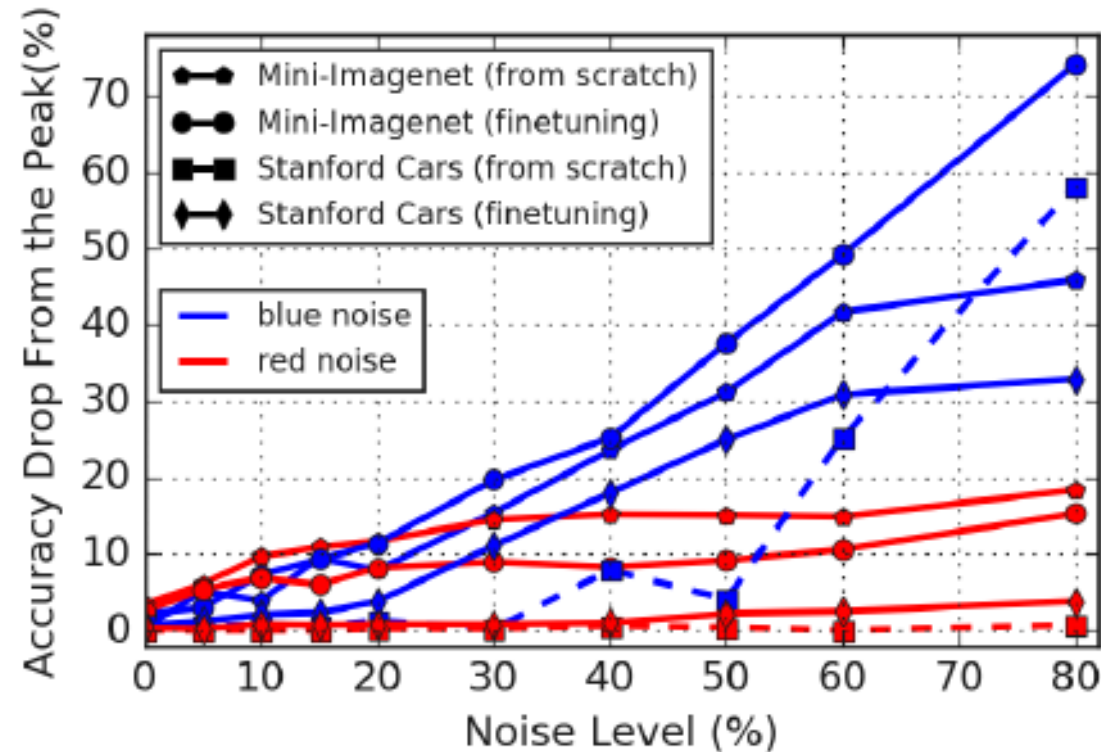
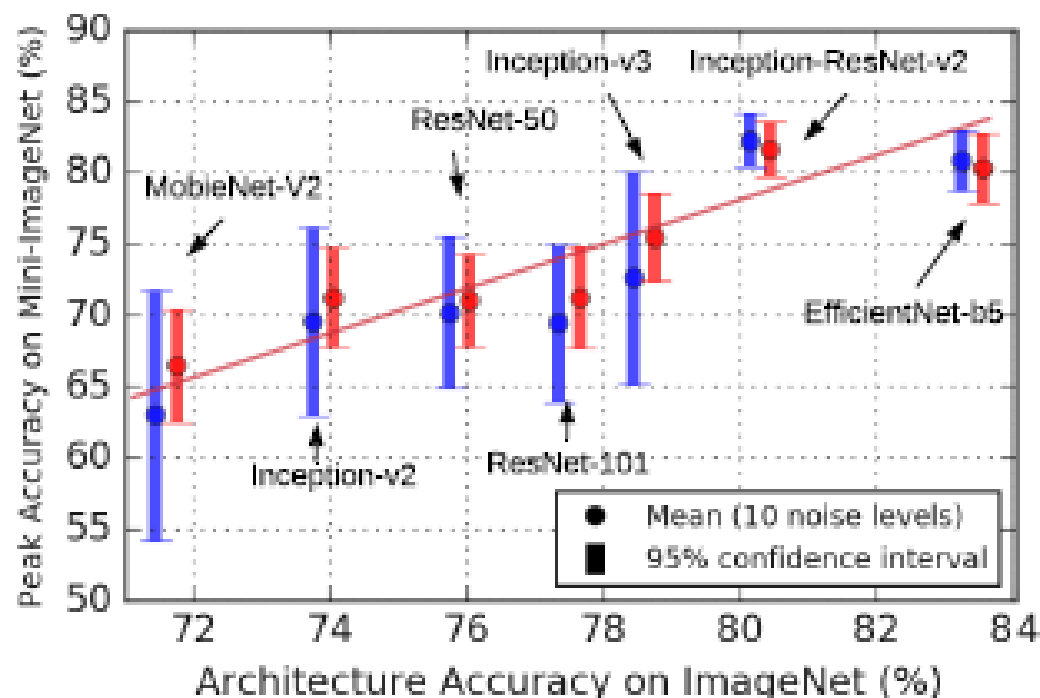
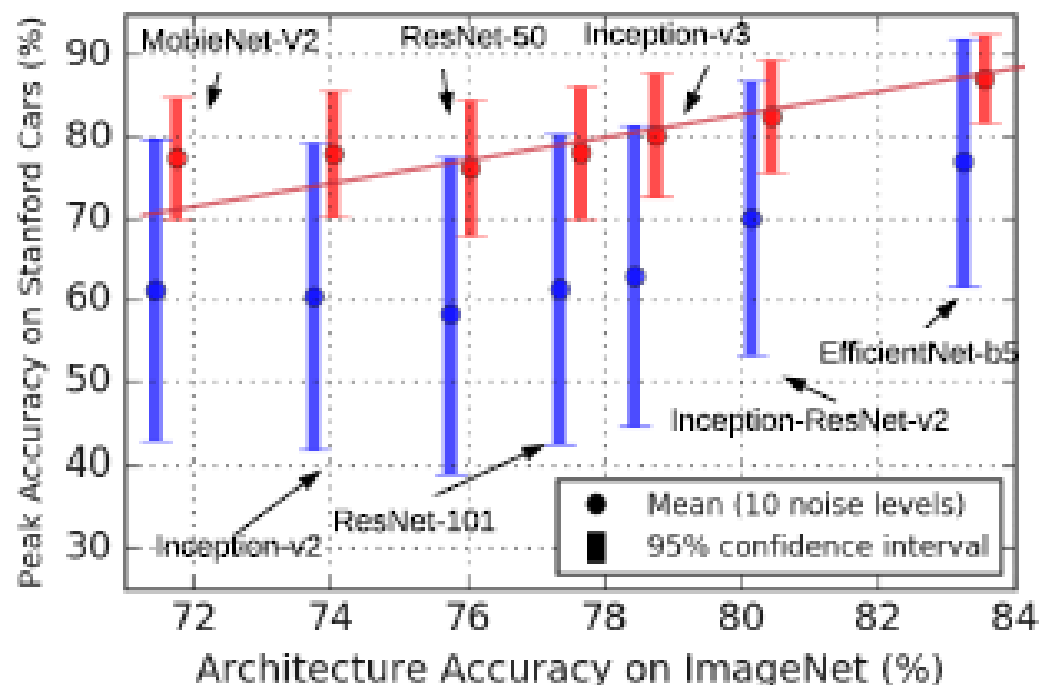


Figure 3. Performance drop from the peak accuracy at different noise levels. Colors are used to differentiate noise types.

Understanding DNNs trained on noisy labels



(a) Mini-ImageNet ($r = 0.91$)



(b) Stanford Cars ($r = 0.88$)

Thank you